Smart Irrigation and Monitoring System using ESP32 and RTL/SoC Design

A

Summer Internship (II) Report

Submitted By

Ayush Tiwari (23116022)

Submitted in the partial fulfillment of the requirements for the subject

Summer Internship (II) in VII semester

of

Bachelor of Technology

in

Electronics & Communication Engineering

to the



Department of Electronics and Communications

National Institute of Technology Raipur

G. E. Road Raipur-492010(C.G.)

November 2026

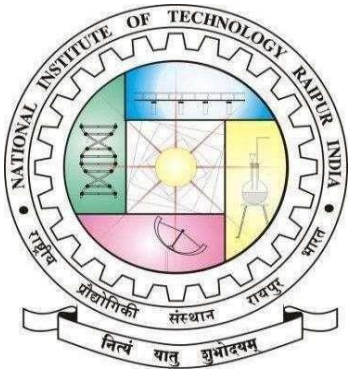
NATIONAL INSTITUTE OF TECHNOLOGY RAIPUR

Certificate by the Examiner

This is to certify that the Summer Internship(II) work entitled "**Smart Irrigation and Monitoring System using ESP32 and RTL/SoC Design**" is the bona-fide work done by **Ayush Tiwari (23116022)** under our guidance and supervision. This report is submitted following the completion of Summer Internship(II) during the academic session of January-May 2026.

Examiner 1	Examiner 2
Name:	**Name:**
Date:	**Date:**

NATIONAL INSTITUTE OF TECHNOLOGY RAIPUR



Declaration

I certify that

a) The work contained in this report is original and has been done by me during the summer vacation.

b) The work has not been submitted to any other institute for any degree or diploma.

c) I have followed the guidelines provided by the Institute in writing the report.

d) Whenever I have used materials (data, theoretical analysis, and text) from other sources, I have given due credit to them in the text of the report and giving their details in the references.

(Signature)
Ayush Tiwari
Roll No.: 23116022
B.Tech (Semester: VII)
Dept. of ECE

ABSTRACT

This internship project presents the design and implementation of a Smart Irrigation and Monitoring System developed in two complementary phases. Phase I is an ESP32-based Internet of Things (IoT) prototype that senses soil moisture, temperature, humidity, rainfall and water-tank level and automatically drives a relay-controlled pump. Remote monitoring, historical graphs and manual override are provided through the Blynk cloud platform on both web and mobile.

Phase II re-implements the irrigation decision algorithm as a modular Verilog RTL design consisting of an Environment Monitor, a Decision Logic unit, a Generic Timer, an Irrigation Finite State Machine and a top-level module. The design was verified with a self-checking testbench (8 of 8 tests passed, 100% FSM state coverage), synthesized in AMD Vivado (25 LUTs and 12 flip-flops), and implemented as an ASIC through the OpenLane 2 RTL-to-GDSII flow on the SKY130 process, closing setup and hold timing at 5 ns across nine PVT corners with zero DRC, LVS and antenna violations. Gate-level simulation of the final netlist matches the RTL.

The project demonstrates a hardware–software co-design path, from an IoT prototype to verified and laid-out digital hardware, for the same agricultural automation problem.

Index Terms—Smart irrigation, ESP32, IoT, Blynk, Verilog HDL, RTL design, finite state machine, Vivado, OpenLane, SKY130, FPGA, SoC.

(Signature)
Ayush Tiwari
Roll No.: 23116022
B.Tech (Semester: VII)
Dept. of ECE

Acknowledgement

I would like to express my sincere gratitude to our faculty incharge training & placement and internship: **Dr. T. Meenpal**, Professor, NIT Raipur for his precious guidance and kind co-operation at every step of this training/internship work. I/we consider myself fortunate to have obtained his friendly and valuable advice during my work.

My sincere thanks to **Dr. B. Patnaik**, Head of the Department, ECE, NIT Raipur for giving me permission to carry out this training/internship.

I am thankful to all the teaching and non-teaching faculty members of ECE department, NIT Raipur for helping me during this project.

I appreciate the help extended by all the team members of **Motilal Nehru National Institute of Technology Allahabad** for helping during training/internship period.

Finally, I am very thankful to my family members and friends for their great moral support.

(Signature)
Ayush Tiwari
Roll No.: 23116022
B.Tech (Semester: VII)
Dept. of ECE

Table of Contents

ABSTRACT IV
Acknowledgement V
List of Figures IX
List of Tables XI
List of Symbols / Abbreviations XII
1. INTRODUCTION 1
1.1 Objectives 1
1.2 Scope of the Project 2
2. ORGANIZATION PROFILE 3
3. LITERATURE SURVEY 4
3.1 Traditional Irrigation Systems 4
3.2 IoT-Based Smart Irrigation 4
3.3 ESP32 in Embedded Systems 4
3.4 Role of FPGA and ASIC in Agriculture-Oriented Embedded Systems 4
3.5 Research Gap 5
3.6 Proposed Solution 5
4. SYSTEM ARCHITECTURE AND HARDWARE DESIGN 6
4.1 Overall System Architecture 6
4.2 Hardware Components 7
4.2.1 ESP32 Development Board 7
4.2.2 Capacitive Soil Moisture Sensor 7
4.2.3 DHT11 Temperature and Humidity Sensor 7
4.2.4 Rain Sensor 7
4.2.5 Ultrasonic Sensor (HC-SR04) 7
4.2.6 Relay Module 8
4.2.7 Water Pump 8
4.2.8 Power Supply 8
4.2.9 Hardware Specification Summary 8
4.3 Hardware Connections 9
5. FIRMWARE DESIGN AND CLOUD INTEGRATION 11
5.1 Firmware Modules 11
5.2 Automatic Irrigation Algorithm 11
5.3 Manual Mode 13
5.4 Cloud Integration 13

5.4.1 Arduino Serial Monitor 14
5.4.2 Blynk Web Dashboard 15
5.4.3 Event Configuration 16
5.4.4 Mobile Application Dashboard 16
6. RTL/SoC DESIGN OF THE IRRIGATION CONTROLLER 19
6.1 RTL/SoC Design Methodology 19
6.1.1 RTL Design Flow 19
6.2 Environment Monitor Module 20
6.2.1 Purpose 20
6.2.2 Interface 20
6.2.3 Working Principle 21
6.2.4 Advantages of This Approach 21
6.3 Decision Logic Module 21
6.3.1 Interface 22
6.3.2 Priority Logic 22
6.4 Generic Timer 24
6.4.1 Features 24
6.4.2 Working 24
6.4.3 Advantages 24
6.5 Irrigation FSM 24
6.5.1 States 25
6.5.2 State Transition Table 25
6.5.3 Outputs and Diagnostics 26
6.6 Top Module Integration 26
6.6.1 Top-Level Ports 26
6.6.2 Why a Modular Architecture Was Preferred 27
7. VERIFICATION AND IMPLEMENTATION OF THE RTL DESIGN 28
7.1 Functional Verification 28
7.1.1 Verification Methodology 28
7.1.2 Test Scenarios 28
7.1.3 Simulation Result 29
7.2 Simulation Waveform Analysis 29
7.2.1 Waveform 1 — Manual Mode, Fault, and Coverage Tracking 30
7.2.2 Waveform 2 — Automatic Cycle, Rain Inhibition, and Manual Recovery 30
7.3 Synthesis and Implementation Results (Vivado) 31
7.3.1 Synthesized Schematic 31
7.3.2 Resource Utilization 32

7.3.3 Timing Summary 33
7.3.4 Device Floorplan 33
7.4 Physical Design Implementation using OpenLane 2 34
7.4.1 Objective and Flow Overview 34
7.4.2 Design Preparation 34
7.4.3 Timing Constraints 35
7.4.4 Effect of the Synthesis Strategy 35
7.4.5 Clock-Period Sweep 36
7.4.6 Final Implementation Results (5 ns, DELAY 0) 37
7.4.7 Post-Layout (Gate-Level) Verification 39
7.4.8 Observations and Limitations 40
7.4.9 Summary 40
8. KEY LEARNINGS AND SKILLS ACQUIRED 42
8.1 Technical Knowledge 42
8.1.1 Embedded Systems and IoT Development 42
8.1.2 Digital Design and RTL Modelling 42
8.1.3 Functional Verification 42
8.1.4 Synthesis and Physical Design 43
8.2 Practical and Analytical Skills 43
8.3 Professional Competencies 43
9. RESULTS AND DISCUSSION 44
9.1 Summary of Key Results 44
9.2 Comparison Between IoT and RTL Implementations 44
10. CONCLUSION AND FUTURE SCOPE 46
10.1 Major Achievements 46
10.2 Future Work 46
APPENDICES 49
Appendix A – Complete Arduino Source Code (test.ino) 50
Appendix B – environment_monitor.v 54
Appendix C – decision_logic.v 55
Appendix D – generic_timer.v 57
Appendix E – irrigation_fsm.v 59
Appendix F – smart_irrigation_top.v 63
Appendix G – tb_smart_irrigation.v 65

List of Figures

Figure 4.1: Overall IoT system block diagram (Phase I) — sensor inputs feed the ESP32, which drives the relay/pump and synchronizes with the Blynk cloud for remote monitoring and control. 6

Figure 4.2: Assembled ESP32-based hardware setup on breadboard, showing the microcontroller, relay module, and sensor wiring. 8

Figure 4.3: Rain sensor and irrigation delivery tube mounted on the plant pot. 10

Figure 4.4: HC-SR04 ultrasonic sensor mounted on the rim of the water storage tank for water-level sensing. 10

Figure 5.1: Firmware automatic-irrigation algorithm flowchart. 13

Figure 5.2: Arduino IDE serial monitor showing live moisture, temperature, humidity, rain, tank level, and pump status output. 14

Figure 5.3: Blynk web dashboard in automatic monitoring mode — moisture 61%, temperature 30.5°C, humidity 88%, tank level 78%, pump idle. 15

Figure 5.4: Blynk web dashboard with manual mode enabled and the pump actively running (Pump Status indicator lit). 15

Figure 5.5: Blynk console – Events & Notifications configuration for Tank Empty and Low Soil Moisture conditions. 16

Figure 5.6: Blynk mobile app – Plantation_Node_1 live dashboard with gauges and manual controls. 17

Figure 5.7: Blynk mobile app – historical graph view of moisture, temperature, humidity, and tank level over time. 18

Figure 6.1: RTL design flow followed for the hardware phase of the project. 20

Figure 6.2: Decision Logic priority flow, exactly as implemented in the always block of decision_logic.v. 23

Figure 6.3: Irrigation FSM state diagram, derived directly from the next-state logic in irrigation_fsm.v. 25

Figure 6.4: Vivado elaborated RTL schematic of smart_irrigation_top, showing the ENV_MON, DECISION, TIMER, and FSM instances and their interconnecting nets. (Captured before the unused inhibit_reason input of the FSM was removed during the OpenLane preparation in Section 7.4; in the final RTL, inhibit_reason is driven only to the top-level output by the Decision Logic.) 27

Figure 7.1: Vivado simulator – waveform panel and Tcl console showing the final self-checking testbench summary (Tests Passed: 8, Tests Failed: 0, ALL TESTS PASSED) together with full FSM state coverage. 29

Figure 7.2: Simulation waveform showing environmental input transitions, manual-mode override, FAULT-state coverage (visited_fault, shown in red), and the pass_count/fail_count bookkeeping signals. 30

Figure 7.3: Post-synthesis schematic of smart_irrigation_top (63 Cells, 54 I/O Ports, 107 Nets). 32
Figure 7.4: Vivado Utilization Summary report. 32
Figure 7.5: Vivado Design Timing Summary — Setup, Hold, and Pulse Width reports. 33
Figure 7.6: Device view showing the physical placement footprint of the synthesized design on the xcve2002-sbva484-1LHP-i-L device. 34
Figure 7.7: Final routed layout of smart_irrigation_top (120 × 120 µm die) viewed in KLayout, showing the local-interconnect, metal-1, via and metal-2 layers. The 54 I/O pins are placed along the die edges. 38
Figure 7.8: Zoomed view of the standard-cell area showing metal-1 power rails, cell-level routing on metal-1 and the vertical metal-2 routes. 39

List of Tables

Table 1: Hardware component specifications 8
Table 2: Complete pin mapping (as defined in the Arduino firmware) 9
Table 3: Blynk virtual pin mapping 14
Table 4: Environment Monitor threshold parameters (as instantiated with default values) 21
Table 5: Environment Monitor truth table (– indicates the condition is independent of that column) 21
Table 6: inhibit_reason encoding (localparam values from decision_logic.v) 23
Table 7: env_status[3:0] bit assignment ({rain, low_humidity, hot, dry}) 23
Table 8: FSM state encoding (localparam values from irrigation_fsm.v) 25
Table 9: FSM state transition table, exactly matching the case statement in irrigation_fsm.v 25
Table 10: Self-checking testbench scenarios (eight check_output calls in total, matching the eight tests reported below) 28
Table 11: Post-synthesis resource utilization summary 32
Table 12: OpenLane 2 configuration of the final run (parameters not listed were left at their defaults) 35
Table 13: Effect of the ABC synthesis strategy on the synthesized netlist (6 ns runs, identical repair margins) 36
Table 14: Clock-period sweep (worst slack over all nine PVT corners; DRC, LVS and antenna checks were clean in every run) 37
Table 15: Summary of the final implementation 37
Table 16: Worst setup and hold slack in each of the nine analysis corners (final run) 38
Table 17: Summary of key results of the project 44
Table 18: IoT firmware vs. RTL hardware implementation comparison 44

List of Symbols / Abbreviations

Symbol / Abbreviation	Description
ADC	Analog-to-Digital Converter
ASIC	Application-Specific Integrated Circuit
CTS	Clock Tree Synthesis
DRC	Design Rule Check
ESP32	Wi-Fi/Bluetooth System-on-Chip microcontroller by Espressif Systems
FF	Flip-Flop
FPGA	Field-Programmable Gate Array
FSM	Finite State Machine
GDSII	Graphic Data System II (layout stream format)
GPIO	General-Purpose Input/Output
HDL	Hardware Description Language
IoT	Internet of Things
LUT	Look-Up Table
LVS	Layout Versus Schematic
PDK	Process Design Kit
PVT	Process, Voltage and Temperature
RTL	Register Transfer Level
SDC	Synopsys Design Constraints
SKY130	SkyWater 130 nm open-source process
SoC	System-on-Chip
STA	Static Timing Analysis
WNS / WHS	Worst Negative Slack / Worst Hold Slack

1. INTRODUCTION

Agriculture accounts for a significant share of global freshwater consumption, and conventional irrigation practices which typically rely on fixed schedules or manual observation, often lead to overwatering, underwatering, or unnecessary water wastage. Smart irrigation systems address these inefficiencies by combining environmental sensing, embedded control, and automated decision-making to apply water only when and where it is actually needed.

This project implements such a system in two stages. In the first stage, an ESP32 microcontroller acts as the central IoT platform: soil moisture, temperature, humidity, rainfall, and water tank level are continuously sensed, and irrigation is automatically started or stopped based on predefined thresholds. The system also allows the user to remotely observe all sensor parameters and manually override pump operation through the Blynk cloud application, from either a web dashboard or a mobile app.

In the second stage, the project moves beyond a purely software-based implementation by redesigning the same irrigation control algorithm at the Register Transfer Level (RTL) using Verilog HDL. This demonstrates how a control algorithm first validated in embedded C++ can be re-expressed as synthesizable digital hardware suitable for FPGA prototyping and, ultimately, ASIC implementation, a practical illustration of hardware–software co-design [1].

1.1 Objectives

- Design and build an automated, sensor-driven irrigation controller using the ESP32 platform.
- Enable cloud-based remote monitoring and manual pump control through the Blynk platform (web and mobile).
- Develop reliable automatic irrigation logic that accounts for soil moisture, temperature, humidity, rainfall, and tank level.
- Re-implement the core control algorithm as a modular RTL design in Verilog HDL.
- Verify the RTL design through simulation using a self-checking testbench, and synthesize it in AMD Vivado.

- Compare the IoT (software) and RTL (hardware) implementations of the same control problem.

1.2 Scope of the Project

The scope of this internship covers the complete development cycle of both implementation phases, from sensor interfacing and firmware development through to RTL coding, functional verification, logic synthesis, and ASIC physical implementation (RTL to GDSII, Section 7.4), but stops short of physical FPGA board deployment or ASIC fabrication, both of which are identified as future work in Chapter 10.

2. ORGANIZATION PROFILE

Name of Organization: Innovation and Incubation Hub MNNIT Foundation (IIHMF), Motilal Nehru National Institute of Technology (MNNIT) Allahabad

Location: Barrister Mullah Colony, MNNIT Allahabad Campus, Teliarganj, Prayagraj, Uttar Pradesh 211004

About the Organization: The Innovation and Incubation Hub MNNIT Foundation (IIHMF) is the official incubator of Motilal Nehru National Institute of Technology (MNNIT) Allahabad, located in Prayagraj, Uttar Pradesh [2]. It acts as the central innovation platform of the institute and supports student-led and faculty-led ventures through incubation, mentoring and access to funding [2].

Mandate and Functions: IIHMF contributes to the growth of entrepreneurship and the startup ecosystem in the region. Its programs are aimed at helping early-stage teams move from an initial idea to a working product and then on to the market [2].

Internship Duration and Supervisor: The summer internship was carried out at IIHMF, MNNIT Allahabad, for four weeks, from 16 June 2026 to 13 July 2026, under the guidance of Prof. Basant Kumar. The Smart Irrigation and Monitoring System described in this report was developed as the internship project.

Relevance to the Internship: IIHMF's focus on turning ideas into working prototypes matches the nature of this project, which takes an agricultural automation idea from a sensor-based IoT prototype to a verified and implemented hardware design.

3. LITERATURE SURVEY

3.1 Traditional Irrigation Systems

Conventional irrigation relies on timer-based sprinkler schedules or manual observation by farm personnel. While simple to implement, these approaches do not respond to actual soil or weather conditions, frequently resulting in water wastage during rainfall or crop stress during unexpectedly dry spells.

3.2 IoT-Based Smart Irrigation

The introduction of low-cost Wi-Fi-enabled microcontrollers has made sensor-driven, remotely monitored irrigation systems practical for small-scale and hobbyist deployments. Typical systems combine a soil moisture sensor with a microcontroller and a relay-driven pump, often layered with a cloud dashboard for remote visibility. This project follows that general pattern while extending it with rainfall detection and tank-level protection.

3.3 ESP32 in Embedded Systems

The ESP32 is a widely adopted System-on-Chip for IoT applications due to its dual-core processor, integrated Wi-Fi and Bluetooth radios, multiple ADC channels, and rich set of GPIOs. Its on-board wireless connectivity removes the need for an external communication module, simplifying cloud-connected embedded designs such as the one implemented in this project [2].

3.4 Role of FPGA and ASIC in Agriculture-Oriented Embedded Systems

While most agricultural automation projects stop at the microcontroller-firmware stage, control algorithms that are timing-critical, need to run in parallel with other logic, or are destined for a dedicated product can benefit from being re-implemented as digital hardware. FPGA prototyping allows such control logic to be verified in parallel, deterministic hardware before committing to a full custom ASIC flow, offering a lower-risk path toward a dedicated SoC-based irrigation controller [3].

3.5 Research Gap

Existing literature and hobbyist projects predominantly treat smart irrigation as a purely software (microcontroller-based) problem. Relatively little practical, openly documented work demonstrates the same irrigation decision logic re-implemented and formally verified as RTL hardware, or compares the two approaches side by side.

3.6 Proposed Solution

This project addresses that gap directly by implementing the identical irrigation decision problem twice — once as ESP32 firmware with cloud connectivity, and once as a modular, self-checking-testbench-verified Verilog RTL design — and by explicitly comparing the two in Section 9.2.

4. SYSTEM ARCHITECTURE AND HARDWARE DESIGN

4.1 Overall System Architecture

The proposed system was developed in two major phases. Phase I implements an Internet of Things (IoT) based smart irrigation controller using the ESP32 microcontroller, while Phase II redesigns the irrigation decision logic at the Register Transfer Level (RTL) using Verilog HDL. The objective is not only to solve an agricultural automation problem but also to demonstrate the transition from an embedded software solution to a hardware-oriented System-on-Chip (SoC) implementation.

The ESP32 continuously acquires environmental information from the capacitive soil moisture sensor, DHT11 temperature and humidity sensor, rain sensor, and ultrasonic water level sensor. Sensor readings are processed according to predefined thresholds, and the controller automatically switches the irrigation pump through a relay module based on the resulting environmental conditions. The system also supports manual operation through the Blynk cloud platform, and all sensor values and pump status are transmitted to the cloud, enabling real-time monitoring from anywhere with an Internet connection.

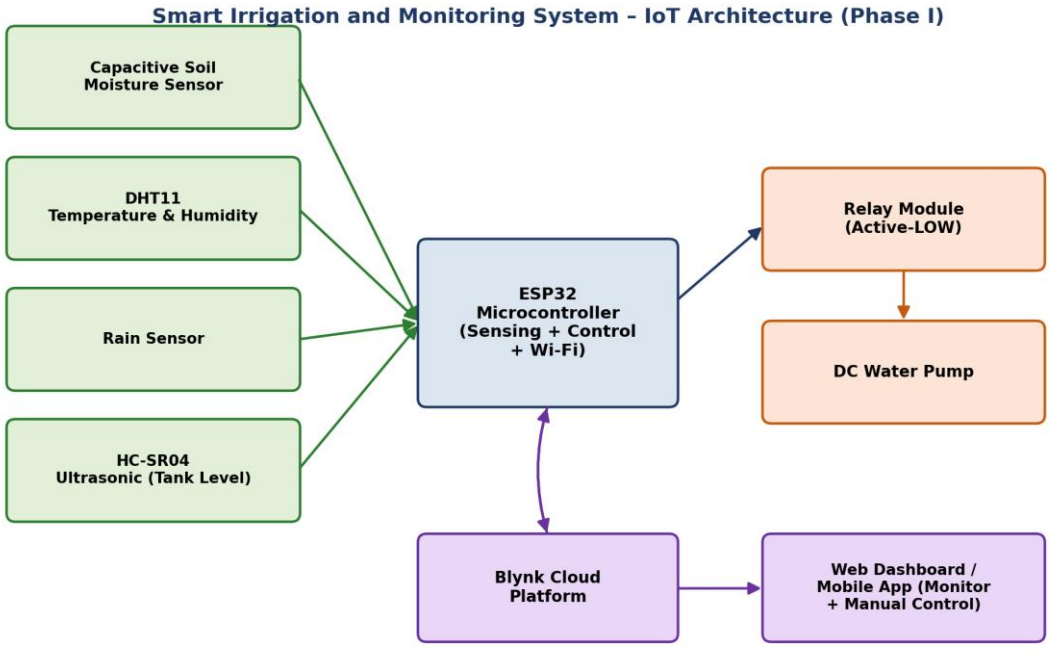


Figure 4.1: Overall IoT system block diagram (Phase I) — sensor inputs feed the ESP32, which drives the relay/pump and synchronizes with the Blynk cloud for remote monitoring and control.

Data flows in one direction from the sensors into the ESP32 for acquisition and processing, and in two directions between the ESP32 and the Blynk cloud: sensor

readings and pump status are pushed to the cloud roughly every two seconds, while manual mode and pump-control commands issued from the dashboard are received back by the ESP32 and acted upon immediately. This closed loop is what allows the system to be monitored and operated remotely, in addition to running its own automatic control algorithm locally.

4.2 Hardware Components

4.2.1 ESP32 Development Board

The ESP32 was selected as the central controller because it integrates a dual-core processor, Wi-Fi and Bluetooth radios, multiple ADC channels, PWM outputs, and a generous set of GPIOs on a single low-cost module. Its built-in wireless connectivity eliminates the need for an external communication module to reach the Blynk cloud.

4.2.2 Capacitive Soil Moisture Sensor

A capacitive sensor was used instead of a resistive type because it offers better corrosion resistance and improved long-term stability when left in contact with damp soil. Its analog output is read on one of the ESP32's ADC-capable pins.

4.2.3 DHT11 Temperature and Humidity Sensor

The DHT11 measures ambient temperature and relative humidity over a single-wire digital interface. Although less precise than the DHT22, it is inexpensive and more than adequate for demonstrating environmental monitoring in this project [4].

4.2.4 Rain Sensor

The rain-sensing module detects the presence of water droplets on its exposed conductive plate and outputs a digital LOW when rainfall is detected. During rainfall, automatic irrigation is inhibited to avoid wasting water.

4.2.5 Ultrasonic Sensor (HC-SR04)

An HC-SR04 ultrasonic sensor continuously measures the distance between itself and the water surface inside the storage tank. This distance is converted into a water-level percentage in firmware, which is in turn used to protect the pump from running dry [5].

4.2.6 Relay Module

A single-channel relay module isolates the ESP32's low-voltage logic from the higher-current pump circuit, acting as the electrical interface between the two.

4.2.7 Water Pump

A miniature DC submersible pump draws irrigation water from the storage tank and delivers it to the plant pot through flexible tubing.

4.2.8 Power Supply

The ESP32 is powered through its USB interface, while the relay and pump draw current from a separate, external regulated supply to keep switching transients away from the microcontroller's power rail.

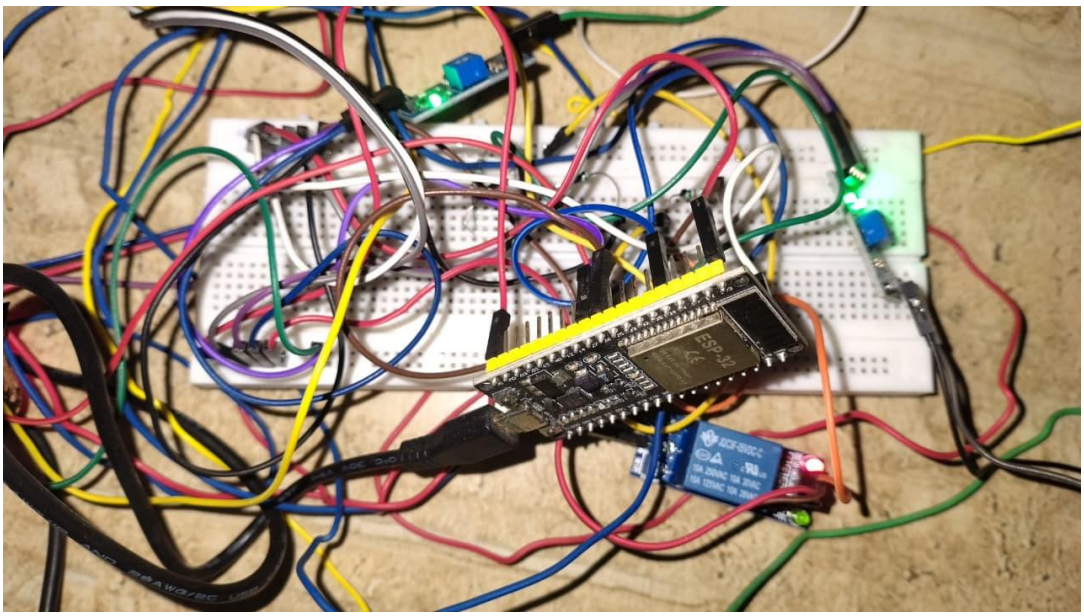


Figure 4.2: Assembled ESP32-based hardware setup on breadboard, showing the microcontroller, relay module, and sensor wiring.

4.2.9 Hardware Specification Summary

Table 1: Hardware component specifications

Component	Type / Model	Function in the System
Microcontroller	ESP32 Dev Module	Sensor acquisition, control logic, Wi-Fi/cloud connectivity
Soil Moisture Sensor	Capacitive analog sensor	Measures soil moisture level (raw ADC → %)
Temperature/Humidity Sensor	DHT11	Measures ambient temperature and relative humidity

Component	Type / Model	Function in the System
Rain Sensor	Resistive/conductive rain module	Detects rainfall to inhibit irrigation
Water Level Sensor	HC-SR04 Ultrasonic	Measures distance to water surface → tank level %
Actuation Interface	1-channel Relay Module	Switches pump power under logic-level control
Actuator	DC Submersible Water Pump	Delivers water from tank to plant pot

4.3 Hardware Connections

Every sensor and actuator in the system connects to the ESP32 through a fixed set of GPIO pins, defined directly in firmware. Analog sensors are read through the ESP32's ADC, the DHT11 and HC-SR04 use their respective single-wire and trigger/echo digital protocols, and the relay is driven from a simple digital output pin.

Table 2: Complete pin mapping (as defined in the Arduino firmware)

ESP32 Pin	Connected To	Signal Type
GPIO 34 (ADC)	Capacitive Soil Moisture Sensor – analog output	Analog input
GPIO 4	DHT11 – data line	Digital, single-wire
GPIO 27	Rain Sensor – digital output	Digital input (active LOW)
GPIO 26	Relay Module – control input	Digital output
GPIO 18	HC-SR04 – TRIG	Digital output (pulse)
GPIO 19	HC-SR04 – ECHO	Digital input (pulse width)
3.3 V	Moisture sensor, rain sensor, DHT11, HC-SR04 logic supply	Power
5 V	Relay module and water pump	Power
GND	Common ground — shared across all modules	Ground reference

Signal levels were kept consistent across the design: all sensor logic operates at 3.3 V, matching the ESP32's native I/O level, while the higher-current relay and pump are powered from a separate 5 V rail so that switching noise on the actuator side does not disturb the sensor readings. The moisture sensor is sampled through the ADC as a raw 12-bit value and averaged over ten readings in firmware before being mapped to a 0–100% scale, while the HC-SR04 distance measurement is converted into a tank-level percentage relative to the physical tank height. All modules share a single common ground to ensure valid analog and digital reference levels throughout the circuit.

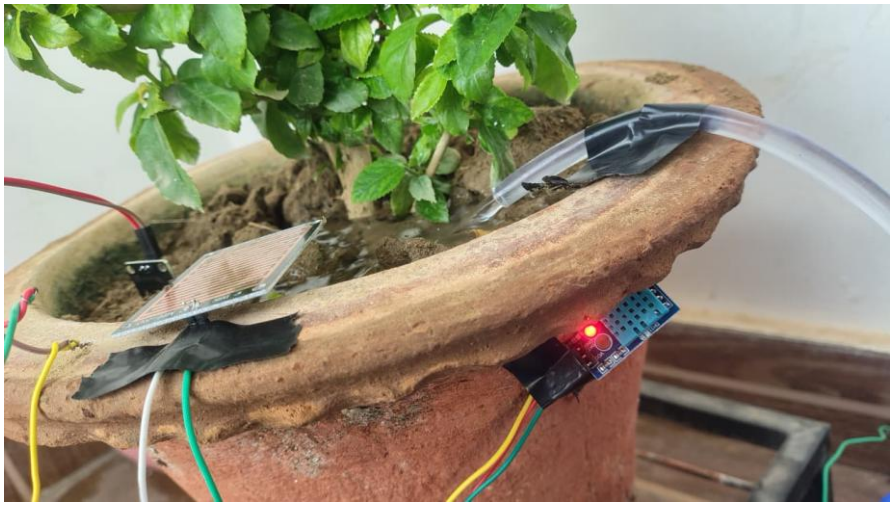


Figure 4.3: Rain sensor and irrigation delivery tube mounted on the plant pot.



Figure 4.4: HC-SR04 ultrasonic sensor mounted on the rim of the water storage tank for water-level sensing.

5. FIRMWARE DESIGN AND CLOUD INTEGRATION

The firmware was developed in the Arduino IDE using Embedded C++ for the ESP32 platform, using the Blynk library for cloud connectivity and the DHT library for sensor access. The sketch is organized around a periodic sensor-and-control task, a set of Blynk virtual-pin callback handlers for remote control, and a background Wi-Fi supervision task.

5.1 Firmware Modules

- Wi-Fi initialization and reconnection supervision.
- Blynk authentication and cloud synchronization.
- Sensor acquisition (moisture, temperature, humidity, rain, tank distance).
- Moisture calibration — raw ADC averaging and mapping to a 0–100% scale.
- Tank-level computation from ultrasonic distance.
- Automatic irrigation decision logic.
- Manual override mode (pump control and mode toggle from the dashboard).
- Cloud data upload and event/notification generation.
- Serial monitor diagnostics.

5.2 Automatic Irrigation Algorithm

The core control task, sendSensorData(), runs on a 2-second Blynk timer and executes the following sequence:

1. Read the soil moisture ADC ten times and average the readings to reduce noise.
2. Trigger the HC-SR04 and compute the tank distance, then convert it into a water-level percentage relative to the configured tank height.
3. Map the averaged moisture reading onto a 0–100% scale using the calibrated DRY_VALUE and WET_VALUE ADC reference points.
4. Read temperature and humidity from the DHT11, and the digital state of the rain sensor.

5. If not in manual mode: turn the pump ON when moisture is below the low threshold, no rain is detected, the tank has sufficient water, and the pump is not already running; turn the pump OFF once moisture rises above the high threshold.
6. If in manual mode and the dashboard pump-control switch is OFF, force the pump OFF regardless of sensor readings.
7. Independently of the mode above, force the pump OFF and raise a tank_empty cloud event whenever the computed water level falls to or below the minimum tank-level threshold.
8. Push moisture, temperature, humidity, rain status, pump status, and tank level to the corresponding Blynk virtual pins, and print a diagnostic line to the serial monitor.

The relevant firmware constants are LOW_THRESHOLD = 20% and HIGH_THRESHOLD = 60% for the moisture hysteresis band, DRY_VALUE = 1700 and WET_VALUE = 900 as the raw-ADC calibration endpoints, TANK_HEIGHT = 20 cm as the physical tank height used for the level calculation, and MIN_TANK_LEVEL = 10% as the safety cut-off below which the pump is disabled and a tank_empty alert is raised.

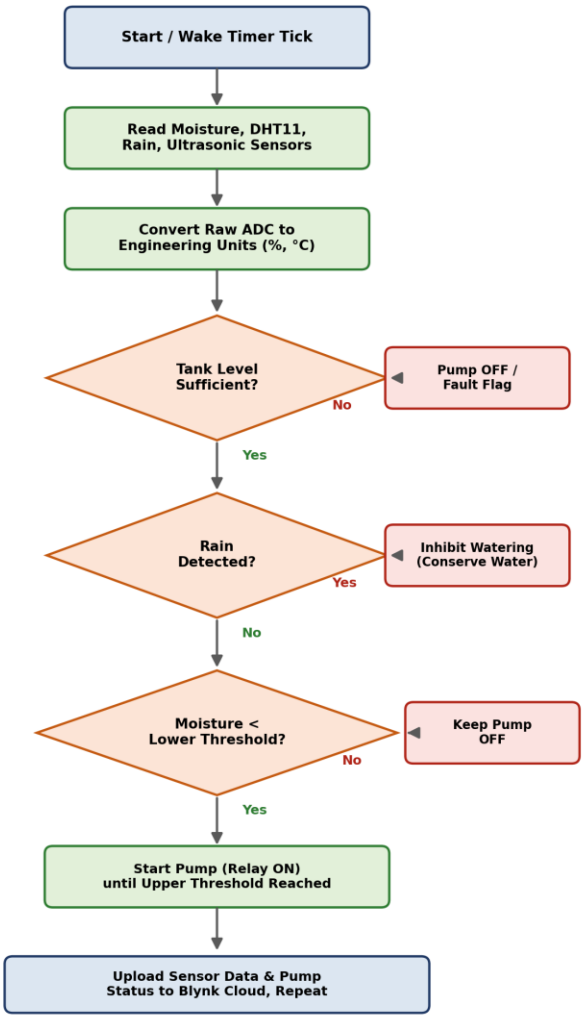


Figure 5.1: Firmware automatic-irrigation algorithm flowchart.

5.3 Manual Mode

Two Blynk virtual pins govern manual operation: V6 toggles manual_mode, and V5 provides the manual pump-control command. When manual mode is enabled, the automatic threshold logic is bypassed entirely and the relay is driven directly by the V5 switch on the dashboard — subject only to the tank-empty safety cut-off, which remains active in both modes.

5.4 Cloud Integration

The Blynk platform provides the real-time dashboards, historical graphs, and remote-control widgets used to monitor and operate the system over the Internet. Sensor values and pump status are written to eight Blynk virtual pins as summarized in Table 3,

and two cloud events (tank_empty and moisture_low) are configured to generate warning notifications directly from the Blynk console [6].

Table 3: Blynk virtual pin mapping

Virtual Pin	Data / Control
V0	Soil moisture (%) — sensor → cloud
V1	Temperature (°C) — sensor → cloud
V2	Relative humidity (%) — sensor → cloud
V3	Rain status (0/1) — sensor → cloud
V4	Pump status (0/1) — sensor → cloud
V5	Manual pump control switch — cloud → actuator
V6	Manual/automatic mode toggle — cloud → firmware
V7	Tank water level (%) — sensor → cloud

Security note: the Wi-Fi credentials and Blynk authentication token used by the firmware are redacted (replaced by placeholder strings) in the code listing of Appendix A.

5.4.1 Arduino Serial Monitor

The serial monitor was used during development to verify raw sensor readings and pump state in real time, independent of the cloud connection.

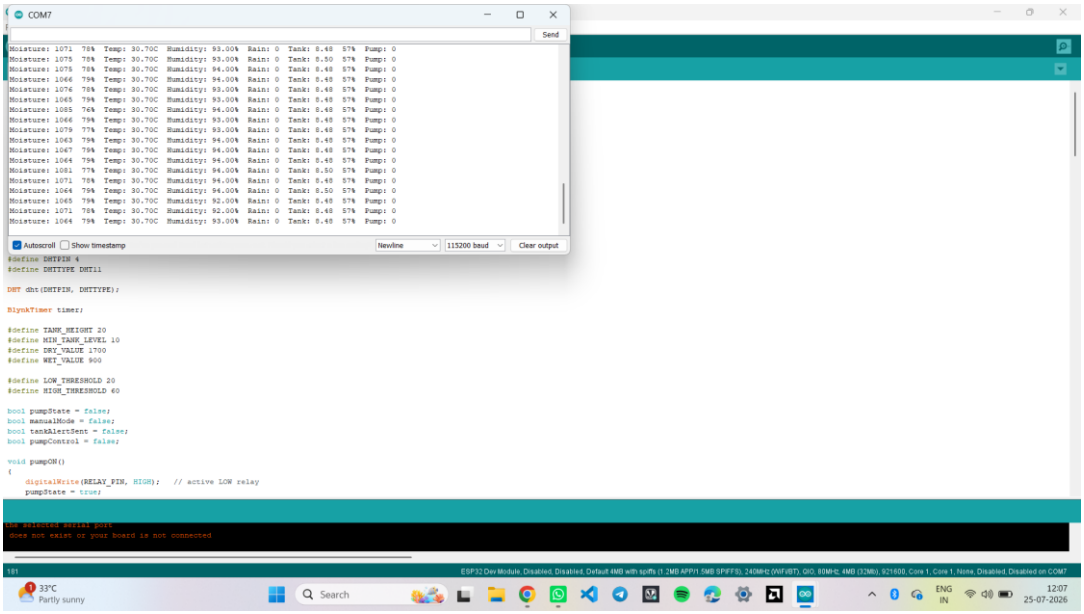


Figure 5.2: Arduino IDE serial monitor showing live moisture, temperature, humidity, rain, tank level, and pump status output.

5.4.2 Blynk Web Dashboard

The web dashboard mirrors the mobile app and provides gauges for moisture, temperature, humidity, and tank level, along with status indicators and control toggles for rain, pump, manual pump control, and mode.

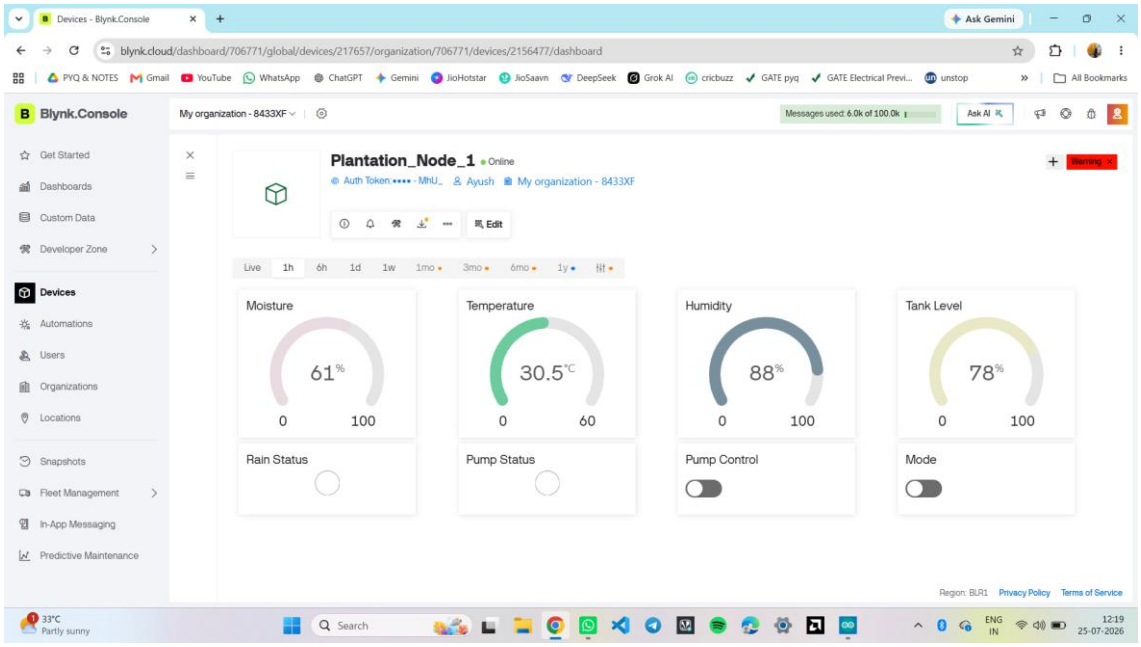


Figure 5.3: Blynk web dashboard in automatic monitoring mode — moisture 61%, temperature 30.5°C, humidity 88%, tank level 78%, pump idle.

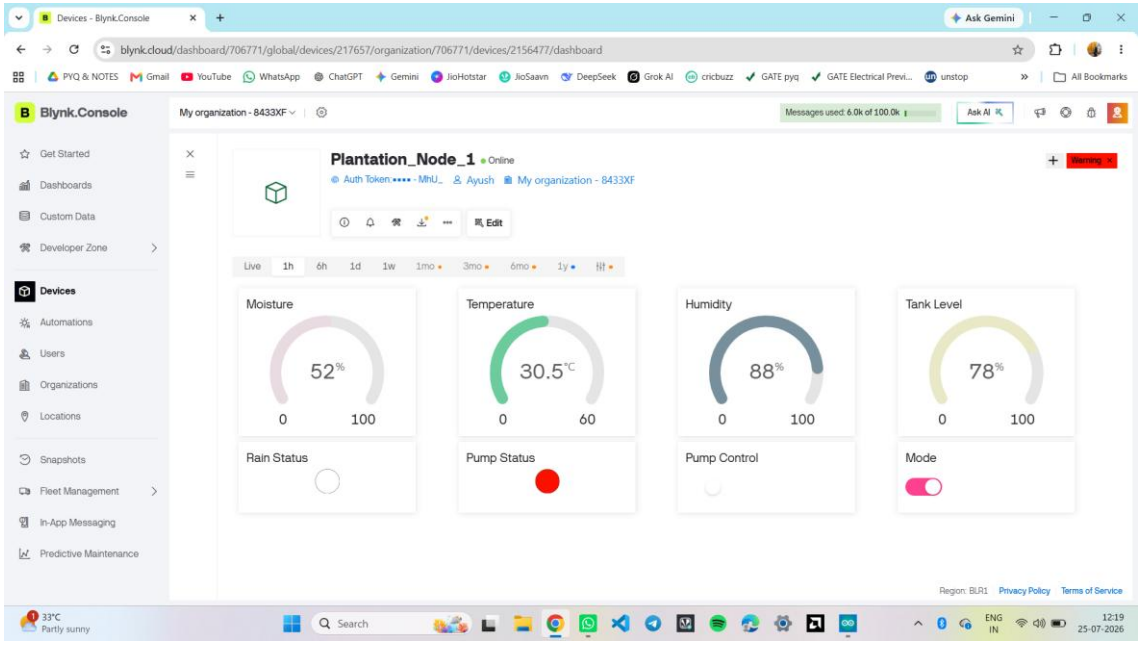


Figure 5.4: Blynk web dashboard with manual mode enabled and the pump actively running (Pump Status indicator lit).

5.4.3 Event Configuration

Two Blynk events were configured on the device template to generate warning-level notifications: tank_empty, triggered when the water level drops to the minimum safe threshold, and moisture_low, triggered on persistently low soil moisture.

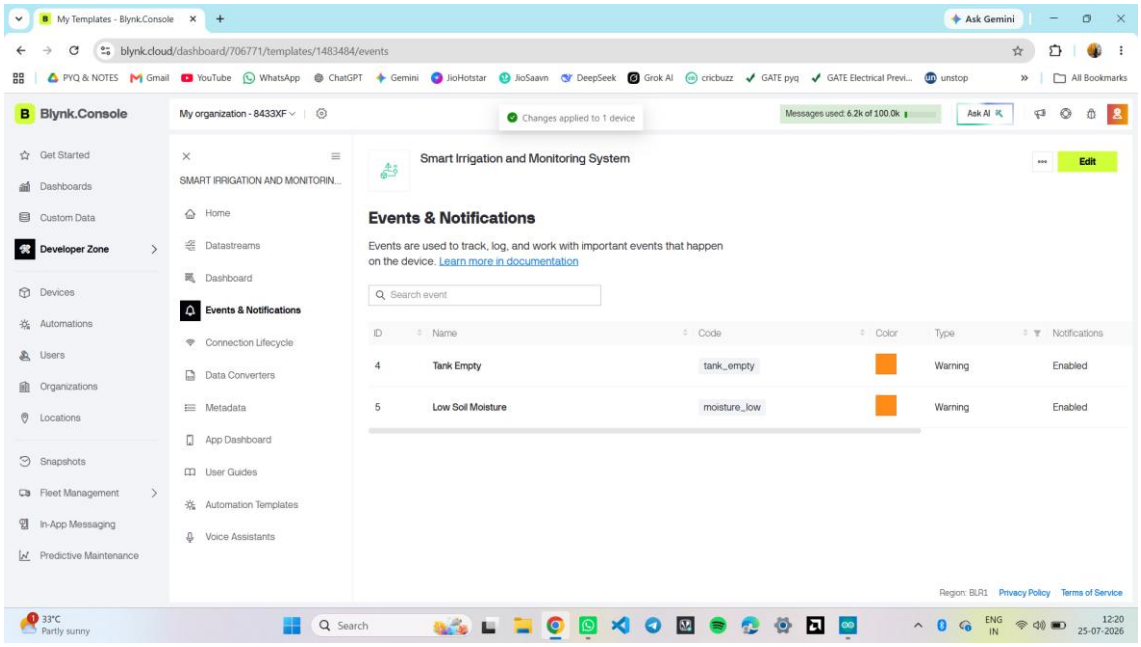


Figure 5.5: Blynk console – Events & Notifications configuration for Tank Empty and Low Soil Moisture conditions.

5.4.4 Mobile Application Dashboard

The Blynk mobile app exposes the same gauges, status indicators, and controls in a layout suited to a phone screen, along with a live historical-trend graph of all four monitored parameters.

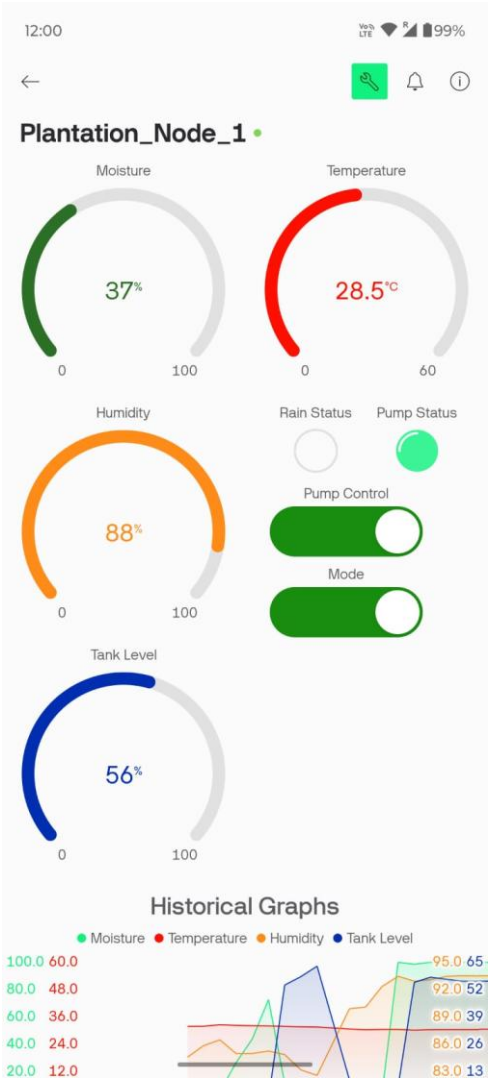


Figure 5.6: Blynk mobile app – Plantation_Node_1 live dashboard with gauges and manual controls.



Figure 5.7: Blynk mobile app – historical graph view of moisture, temperature, humidity, and tank level over time.

6. RTL/SoC DESIGN OF THE IRRIGATION CONTROLLER

6.1 RTL/SoC Design Methodology

The second phase of the project focused on converting the embedded software control algorithm into synthesizable digital hardware. Instead of executing sequential instructions on a microcontroller, the irrigation logic was implemented as interconnected RTL modules using Verilog HDL. The design follows a modular hierarchy, allowing each functional block to be independently verified and then integrated into a complete digital system suitable for FPGA prototyping and future ASIC implementation [7].

The RTL was developed and synthesized in AMD Vivado 2025.2, targeting a Versal xcve2002-sbva484-1LHP-i-L device as the synthesis reference part. Each module was designed with a clearly defined, ANSI-style port interface to simplify both verification and synthesis. A top-level module, smart_irrigation_top, instantiates the Environment Monitor, Decision Logic, Generic Timer, and Irrigation FSM modules and wires them together [8].

6.1.1 RTL Design Flow

1. Behavioural specification of the irrigation control algorithm.
2. Module partitioning into Environment Monitor, Decision Logic, Generic Timer, and FSM blocks.
3. Verilog RTL coding of each module with well-defined ports.
4. Functional simulation of individual modules and the integrated top module.
5. Self-checking testbench verification with automated PASS/FAIL reporting.
6. RTL elaboration to confirm correct module interconnection.
7. Logic synthesis in Vivado.
8. Resource utilization analysis.
9. Timing analysis of the synthesized design.

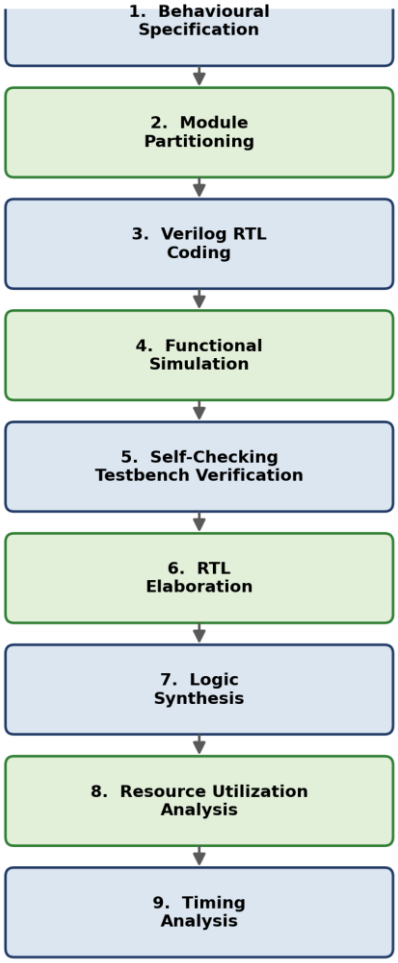


Figure 6.1: RTL design flow followed for the hardware phase of the project.

6.2 Environment Monitor Module

6.2.1 Purpose

The Environment Monitor converts raw 8-bit sensor values into logical environmental conditions. Rather than comparing analogue-style values throughout the rest of the design, this module generates simple binary decision signals that the rest of the pipeline consumes.

6.2.2 Interface

Inputs: moisture[7:0], temperature[7:0], humidity[7:0].

Outputs: dry, wet, hot, low_humidity.

6.2.3 Working Principle

The module compares each sensor reading against a parameterized threshold. If soil moisture is below the LOW_MOISTURE threshold, the dry signal is asserted; if moisture exceeds the HIGH_MOISTURE threshold, wet is asserted instead. Similarly, temperature above HIGH_TEMP asserts hot, and humidity below LOW_HUMIDITY asserts low_humidity. All four comparisons are purely combinational (continuous assign statements), so the outputs update immediately whenever an input changes.

Table 4: Environment Monitor threshold parameters (as instantiated with default values)

Parameter	Value	Meaning
LOW_MOISTURE	20	Moisture below this value → dry = 1
HIGH_MOISTURE	70	Moisture above this value → wet = 1
HIGH_TEMP	30	Temperature above this value → hot = 1
LOW_HUMIDITY	70	Humidity below this value → low_humidity = 1

Table 5: Environment Monitor truth table (– indicates the condition is independent of that column)

Condition	dry	wet	hot	low_humidity
moisture < 20	1	0	–	–
moisture > 70	0	1	–	–
20 ≤ moisture ≤ 70	0	0	–	–
temperature > 30	–	–	1	–
humidity < 70	–	–	–	1

6.2.4 Advantages of This Approach

- Simplifies higher-level decision making — downstream logic reasons about single-bit flags rather than raw sensor magnitudes.
- Reduces combinational complexity in the Decision Logic module.
- Enables independent, modular verification of threshold behaviour.

6.3 Decision Logic Module

The Decision Logic module represents the intelligence of the irrigation controller. It combines the environmental flags produced by the Environment Monitor with two external safety/context signals — rain and tank_empty — to determine whether a watering request should be issued.

6.3.1 Interface

Inputs: dry, hot, low_humidity, rain, tank_empty.

Outputs: watering_request, inhibit_reason[2:0], env_status[3:0].

6.3.2 Priority Logic

The module evaluates conditions in strict priority order inside a single combinational always block, so that exactly one inhibit reason (or none) is reported at any time:

1. Tank Empty — highest priority; irrigation is always blocked when the tank is empty, regardless of any other condition.
2. Rain Detected — irrigation is inhibited to avoid wasting water during rainfall.
3. Temperature Too Low (hot = 0) — irrigation is withheld when conditions are not warm enough to justify watering.
4. Humidity Already High (low_humidity = 0) — irrigation is withheld when ambient humidity is already adequate.
5. Soil Not Dry (dry = 0) — irrigation is withheld when the soil is not actually dry.
6. If none of the above apply, watering_request is asserted and inhibit_reason reports NO_INHIBIT.

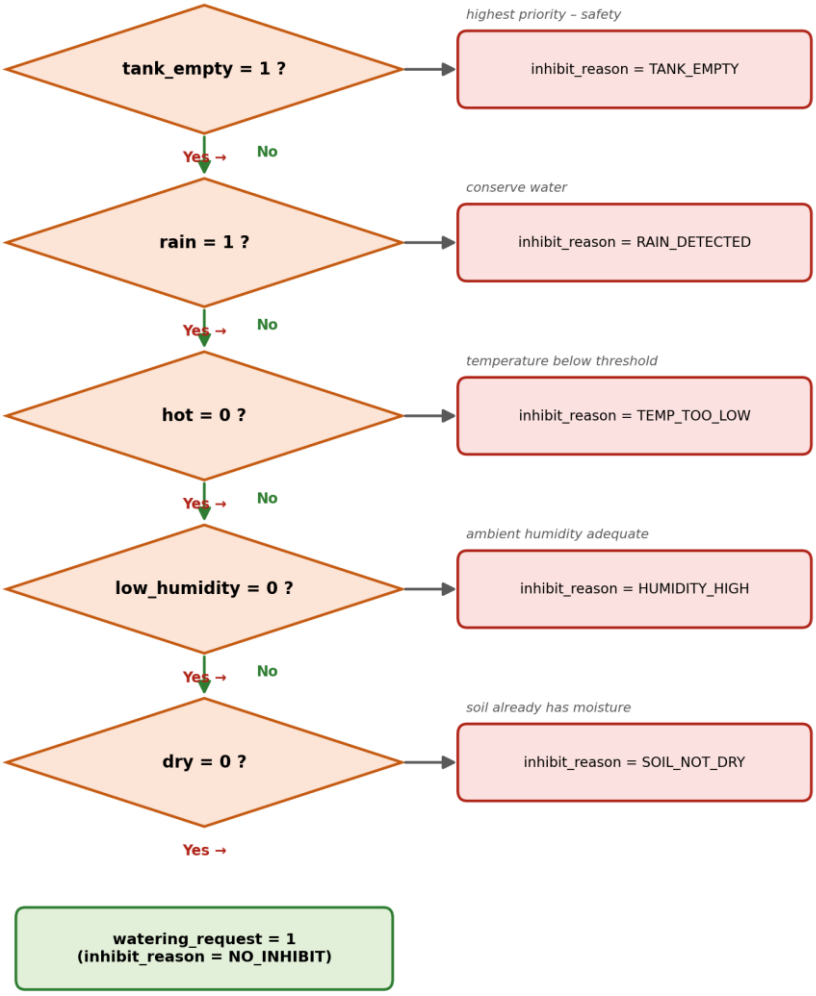


Figure 6.2: Decision Logic priority flow, exactly as implemented in the always block of decision_logic.v.

Table 6: inhibit_reason encoding (localparam values from decision_logic.v)

inhibit_reason	Encoding	Meaning
NO_INHIBIT	3'b000	No blocking condition — watering permitted
RAIN_DETECTED	3'b001	Rain sensor active
TANK_EMPTY	3'b010	Water tank empty
TEMP_TOO_LOW	3'b011	Temperature not above HIGH_TEMP
HUMIDITY_HIGH	3'b100	Humidity not below LOW_HUMIDITY
SOIL_NOT_DRY	3'b101	Moisture not below LOW_MOISTURE

Table 7: env_status[3:0] bit assignment ({rain, low_humidity, hot, dry})

Bit	Signal	Meaning when 1
env_status[3]	rain	Rain currently detected
env_status[2]	low_humidity	Humidity below LOW_HUMIDITY
env_status[1]	hot	Temperature above HIGH_TEMP
env_status[0]	dry	Moisture below LOW_MOISTURE

6.4 Generic Timer

The Generic Timer module ensures that irrigation continues for a predefined duration rather than switching rapidly every time a sensor reading updates, which would otherwise cause the pump to chatter on and off.

6.4.1 Features

- Parameterized counter width (COUNT_WIDTH) and target count (TIMER_COUNT).
- Fully synchronous operation, clocked on the positive edge of clk.
- Single enable input that starts/holds the count.
- Asynchronous reset input.
- Single-bit timer_done output.

6.4.2 Working

While enable is high, the internal counter increments once per clock cycle until it reaches TIMER_COUNT, at which point timer_done is asserted and held. While enable is low, the counter is held at zero and timer_done is deasserted, so the timer restarts cleanly the next time it is enabled. In the top-level instantiation used in this project, the timer is configured with COUNT_WIDTH = 8 and TIMER_COUNT = 100, giving a 100-cycle watering duration once the FSM asserts timer_enable.

6.4.3 Advantages

- Eliminates pump chatter by enforcing a minimum watering duration.
- Improves deterministic, repeatable behaviour during simulation and in the field.
- Fully reusable in other RTL designs thanks to its parameterized width and count.

6.5 Irrigation FSM

The Irrigation FSM is the control unit of the RTL architecture. It is a Moore machine — all outputs (pump, timer_enable, status) depend only on the current state, not on the inputs directly — which receives environmental and timer status and generates the pump control and diagnostic signals.

6.5.1 States

Table 8: FSM state encoding (localparam values from irrigation_fsm.v)

State	Encoding	Description
RESET	3'd0	Entered asynchronously whenever rst is asserted; initializes the controller.
IDLE	3'd1	Default monitoring state; watches for a watering request, a manual command, or a tank fault.
WATER	3'd2	Pump is driven ON; the Generic Timer runs until timer_done, or the state is exited by manual mode or a fault.
FAULT	3'd3	Entered whenever tank_empty is detected in IDLE or WATER; pump is forced OFF until the tank recovers.

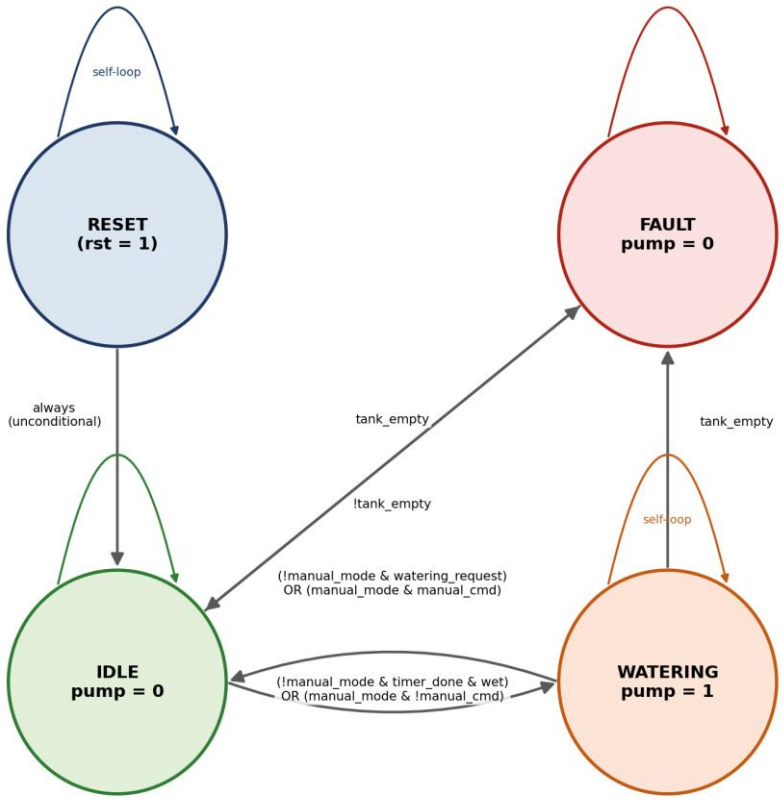


Figure 6.3: Irrigation FSM state diagram, derived directly from the next-state logic in irrigation_fsm.v.

6.5.2 State Transition Table

Table 9: FSM state transition table, exactly matching the case statement in irrigation_fsm.v

Current State	Condition	Next State
RESET	always (one cycle after rst is released)	IDLE
IDLE	tank_empty = 1	FAULT
IDLE	manual_mode = 1 AND manual_cmd = 1	WATER
IDLE	manual_mode = 0 AND watering_request = 1	WATER

Current State	Condition	Next State
IDLE	none of the above	IDLE (self-loop)
WATER	tank_empty = 1	FAULT
WATER	manual_mode = 1 AND manual_cmd = 0	IDLE
WATER	manual_mode = 0 AND timer_done = 1 AND wet = 1	IDLE
WATER	none of the above	WATER (self-loop)
FAULT	tank_empty = 1	FAULT (self-loop)
FAULT	tank_empty = 0	IDLE

6.5.3 Outputs and Diagnostics

In addition to driving pump and timer_enable, the FSM generates a 5-bit status bus that flags IDLE, WATER, and FAULT conditions along with a manual-mode indicator, and an 8-bit diagnostic bus that packs together watering_request, timer_done, tank_empty, rain, low_humidity, hot, wet, and dry into a single word for external observability. Manual mode allows the operator to override the automatic algorithm from IDLE or WATER at any time, while the tank_empty safety check is preserved in every state — it cannot be bypassed even in manual mode.

6.6 Top Module Integration

The smart_irrigation_top module integrates all four RTL blocks described in Sections 6.2–6.5. Signals generated by the Environment Monitor (dry, wet, hot, low_humidity) feed the Decision Logic module, whose watering_request output — together with wet, tank_empty, and the manual-mode signals — drives the Irrigation FSM. The FSM in turn controls the Generic Timer through timer_enable and consumes its timer_done output to time the watering cycle, and ultimately generates the pump output and all diagnostic buses that are exposed at the top level.

6.6.1 Top-Level Ports

Inputs: clk, rst, moisture[7:0], temperature[7:0], humidity[7:0], rain, tank_empty, manual_mode, manual_cmd.

Outputs: pump, status[4:0], state[2:0], inhibit_reason[2:0], env_status[3:0], diagnostic[7:0].

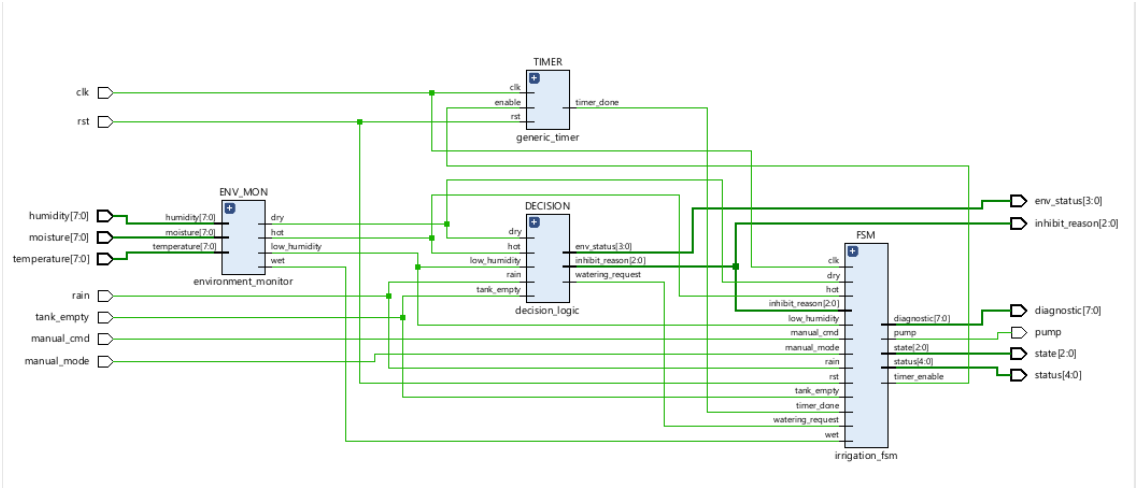


Figure 6.4: Vivado elaborated RTL schematic of smart_irrigation_top, showing the ENV_MON, DECISION, TIMER, and FSM instances and their interconnecting nets. (Captured before the unused inhibit_reason input of the FSM was removed during the OpenLane preparation in Section 7.4; in the final RTL, inhibit_reason is driven only to the top-level output by the Decision Logic.)

In the elaborated schematic of Figure 6.4, the clk and rst inputs fan out to the TIMER and FSM instances (the only two sequential blocks in the design); moisture, temperature, and humidity feed exclusively into ENV_MON; the dry, hot, and low_humidity flags produced by ENV_MON feed both DECISION and FSM directly, since the FSM needs wet and dry information for its own transition and diagnostic logic in addition to what DECISION already consumes; DECISION's watering_request, inhibit_reason, and env_status outputs feed the FSM and/or are routed straight to the top-level outputs; and the FSM's timer_enable output feeds back into TIMER, whose timer_done return path closes the loop.

6.6.2 Why a Modular Architecture Was Preferred

A single monolithic always block containing all of the sensing, decision, timing, and state logic together would have been considerably harder to simulate, debug, and reuse. By contrast, partitioning the design into four independently-instantiated modules with narrow, well-defined interfaces made it possible to verify each block’s combinational or sequential behaviour in isolation before integrating them, kept each module’s internal complexity manageable, and means that, for example, the Generic Timer or Environment Monitor could be reused unchanged in a different control design.

7. VERIFICATION AND IMPLEMENTATION OF THE RTL DESIGN

7.1 Functional Verification

To ensure correct hardware functionality, a self-checking Verilog testbench (tb_smart_irrigation.v) was developed around the smart_irrigation_top DUT. Unlike a conventional testbench that only applies input stimuli for manual waveform inspection, this testbench automatically compares the DUT's pump and state outputs against expected values after every test scenario and reports PASS or FAIL for each one.

7.1.1 Verification Methodology

The testbench is built around three reusable tasks — reset_dut, which pulses rst for three clock cycles; set_environment, which drives moisture, temperature, humidity, rain, and tank_empty and waits one clock edge; and manual_control, which drives manual_mode and manual_cmd — together with a check_output task that compares the DUT's pump and state signals against expected values, incrementing either a pass_count or fail_count integer and printing a [PASS]/[FAIL] line with the mismatch details on failure. A separate always block, running in parallel with the test sequence, sets one of four coverage flags (visited_reset, visited_idle, visited_water, visited_fault) high the first time the FSM enters the corresponding state, giving an independent check that every state was actually exercised during simulation — not just that the final pass/fail count was zero.

7.1.2 Test Scenarios

Table 10: Self-checking testbench scenarios (eight check_output calls in total, matching the eight tests reported below)

#	Test Case	Stimulus	Expected Result
1	RESET	Assert rst for 3 cycles, release	pump = 0, state = IDLE
2	DRY SOIL (automatic watering)	moisture=10, temp=35, hum=40, rain=0, tank_empty=0	pump = 1, state = WATER
3	TIMER	Hold conditions for 110 cycles (≥ TIMER_COUNT), then set moisture=85	pump = 0, state = IDLE
4	RAIN	moisture=10, rain=1, tank_empty=0	pump = 0, state = IDLE
5	TANK EMPTY	moisture=10, rain=0, tank_empty=1	pump = 0, state = FAULT
6	RECOVERY	moisture=85, tank_empty=0	pump = 0, state = IDLE

#	Test Case	Stimulus	Expected Result
7a	MANUAL ON	manual_mode=1, manual_cmd=1	pump = 1, state = WATER
7b	MANUAL OFF	manual_mode=1, manual_cmd=0	pump = 0, state = IDLE

7.1.3 Simulation Result

The simulation completed with all eight self-checks passing and complete FSM state coverage:

- Tests Passed: 8
- Tests Failed: 0
- FSM Coverage: RESET ✓, IDLE ✓, WATERING ✓, FAULT ✓ (100%)

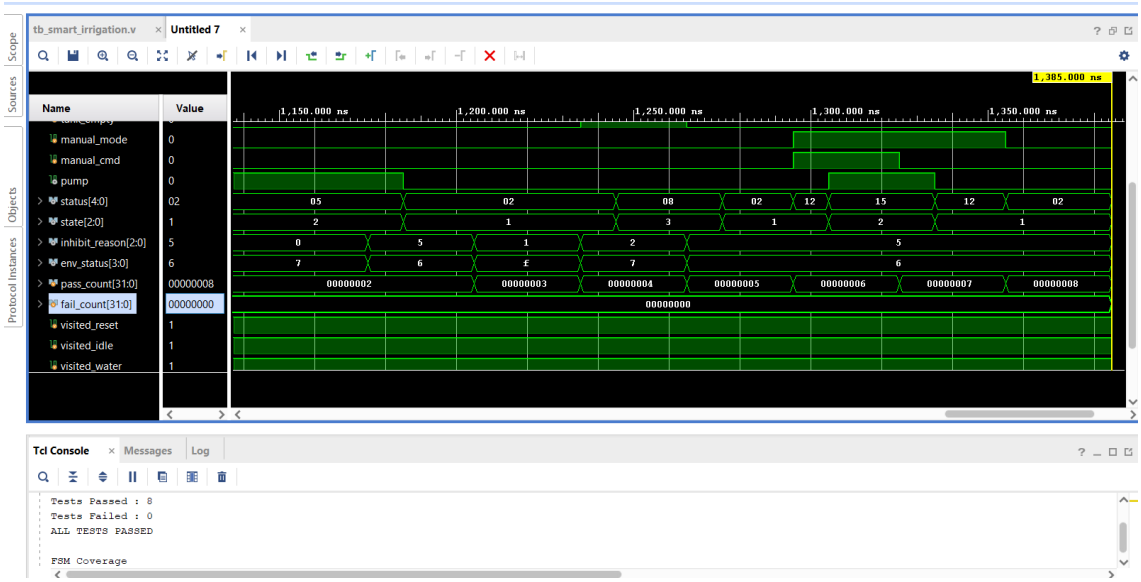


Figure 7.1: Vivado simulator – waveform panel and Tcl console showing the final self-checking testbench summary (Tests Passed: 8, Tests Failed: 0, ALL TESTS PASSED) together with full FSM state coverage.

7.2 Simulation Waveform Analysis

Beyond the pass/fail summary reported by the testbench, the simulation waveforms allow every RTL module's behaviour to be inspected directly. The signal groups of interest are: the clock/reset pair; the environmental inputs (moisture, temperature, humidity, rain, tank_empty); the Environment Monitor's derived flags (dry, wet, hot, low_humidity, visible indirectly through env_status); the Decision Logic outputs (watering_request, env_status, inhibit_reason); the timer's enable/done handshake; and the FSM's state, status, and diagnostic buses, together with the pass_count / fail_count / visited_* bookkeeping signals the testbench maintains.

7.2.1 Waveform 1 — Manual Mode, Fault, and Coverage Tracking

Figure 7.2 captures a window around simulation time 1,200–1,400 ns. The moisture[7:0], temperature[7:0], and humidity[7:0] buses step between the 0x55 and 0x0a test values used by the TIMER and RAIN/RECOVERY scenarios, while the rain and tank_empty pulses (each held high for a short window) can be seen driving the status[4:0] and inhibit_reason[2:0] buses to their corresponding encodings. Later in the same window, manual_mode and manual_cmd are both driven high, and the pump signal responds by going high in step — visibly overriding the automatic decision path. The visited_fault trace, shown in red for the first portion of the capture, confirms that the FAULT state was entered and exercised before the simulation moved on; by the end of the window all four visited_* coverage flags read 1. The fail_count trace briefly reads 00000001 at the point captured in this particular screenshot, corresponding to an intermediate debugging run; the final, authoritative result of the complete regression is the zero-failure summary shown in Figure 7.1.

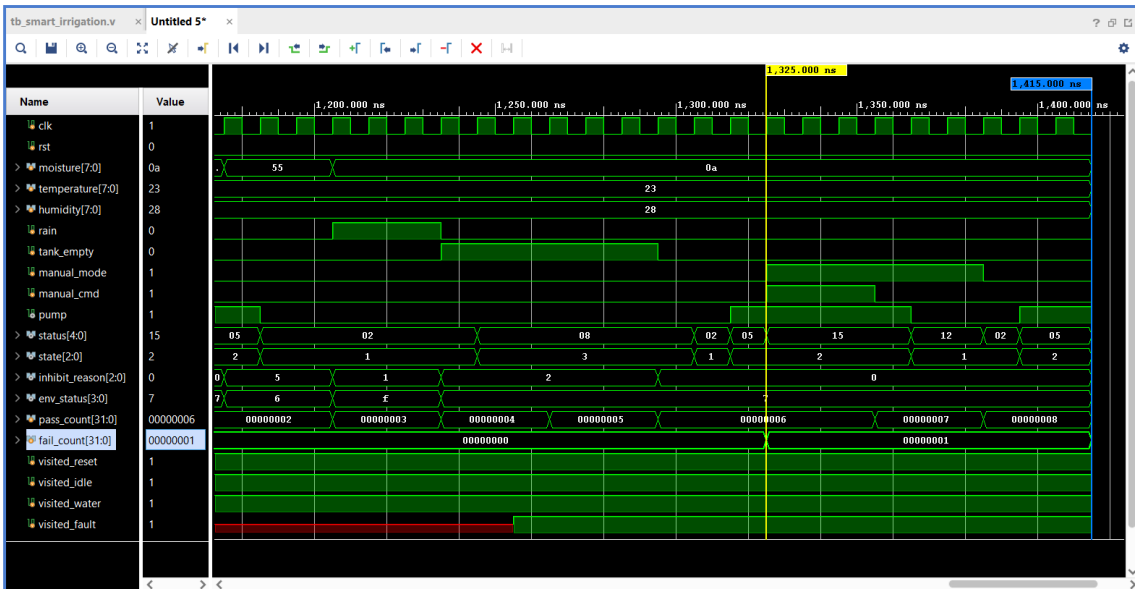


Figure 7.2: Simulation waveform showing environmental input transitions, manual-mode override, FAULT-state coverage (visited_fault, shown in red), and the pass_count/fail_count bookkeeping signals.

7.2.2 Waveform 2 — Automatic Cycle, Rain Inhibition, and Manual Recovery

Figure 7.1's waveform panel (time window approximately 1,150–1,385 ns) shows the same signal set for the tests immediately preceding the final summary: the status[4:0] bus steps through 05 → 02 → 08 → 02 → 12 → 15 → 12 → 02, the state[2:0] bus alternates between IDLE (1) and WATER (2) as the automatic and manual scenarios execute, and inhibit_reason and env_status track the corresponding RAIN,

TANK_EMPTY, and NO_INHIBIT conditions. The pass_count trace increments cleanly from 00000002 through to 00000008 with fail_count remaining at 00000000 throughout this window, directly corroborating the “ALL TESTS PASSED” line printed to the Tcl console at the end of the run.

Taken together, the two captures demonstrate every behaviour required of the controller: automatic watering in response to dry soil, timer-controlled watering duration, rainfall inhibition, tank-empty fault protection, recovery once the tank refills, and manual override — all consistent with the state-transition table of Section 6.5 and the self-checking results of Section 7.1.

7.3 Synthesis and Implementation Results (Vivado)

The verified RTL was synthesized in AMD Vivado 2025.2 against the xcve2002-sbva484-1LHP-i-L reference part. Synthesis converts the behavioural and structural Verilog of Sections 6.2–6.6 into a gate-level netlist of primitive cells (buffers, LUTs, flip-flops, and I/O buffers) while preserving functional equivalence with the elaborated RTL of Figure 6.4.

7.3.1 Synthesized Schematic

Figure 7.3 shows the post-synthesis schematic: 63 cells, 54 I/O ports, and 107 nets. The wide fan-in/fan-out of buffers (BUF, OBUF) visible on the moisture, temperature, and humidity buses reflects the fact that each of these is an 8-bit input that must be individually buffered at the I/O boundary; internally, the TIMER and FSM blocks remain as the only clocked elements, exactly as in the elaborated view, confirming that synthesis optimized the gate-level implementation without altering the design’s functional structure.

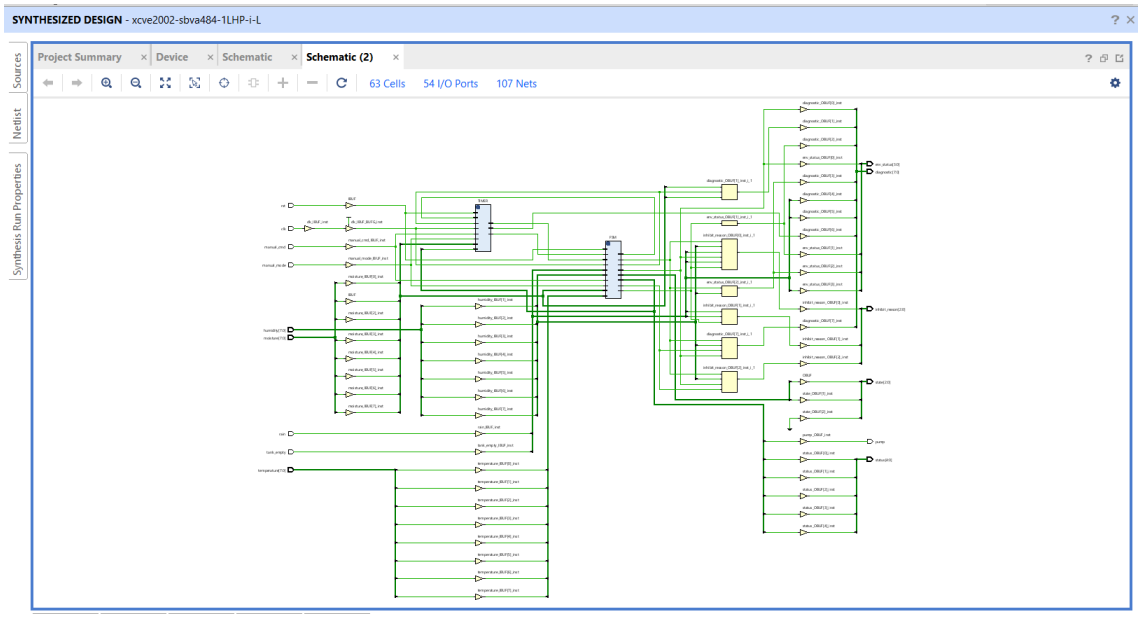


Figure 7.3: Post-synthesis schematic of smart_irrigation_top (63 Cells, 54 I/O Ports, 107 Nets).

7.3.2 Resource Utilization

Table 11: Post-synthesis resource utilization summary

Resource	Utilization	Available	Utilization %
LUT	25	20,000	0.13%
FF (Flip-Flop)	12	40,000	0.03%
I/O	53	162	32.72%

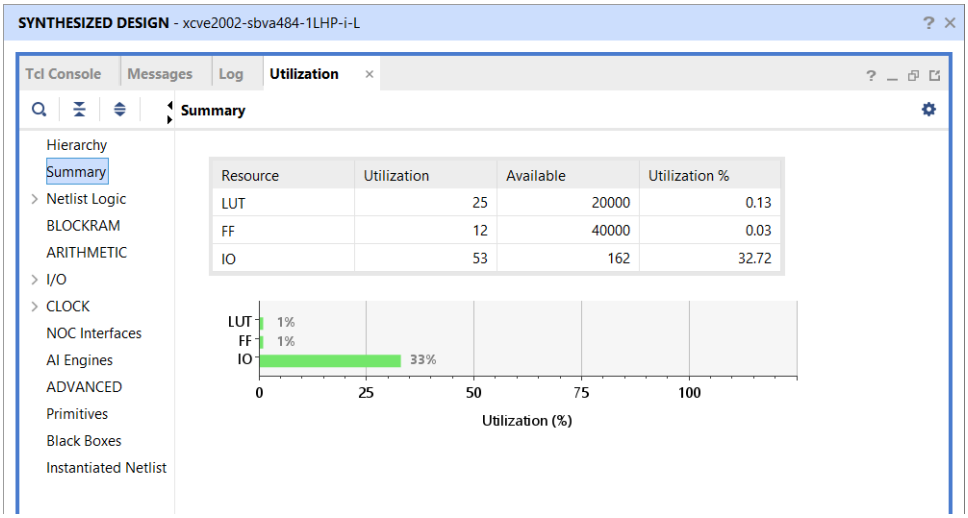


Figure 7.4: Vivado Utilization Summary report.

The low LUT and flip-flop counts reflect the fact that the entire controller is built from simple combinational comparisons, one 8-bit counter, and a 4-state FSM — there is no arithmetic-heavy datapath. The relatively high I/O utilization (32.72%) is a direct consequence of the wide top-level port list (three 8-bit sensor buses plus several

diagnostic buses) rather than any internal logic pressure, and would fall sharply if this controller were integrated as a sub-block of a larger SoC rather than synthesized standalone.

7.3.3 Timing Summary

Since no FPGA-specific timing constraints (a .xdc file) were applied for this standalone synthesis run, Vivado reports a Worst Negative Slack (WNS) and Worst Hold Slack (WHS) of "inf" — meaning there are no user-specified timing paths to fail against — with 0 failing endpoints out of 53 total endpoints for both setup and hold. This confirms that the design synthesizes cleanly and elaborates into valid sequential logic; a full board-level timing closure would require applying a clock constraint and I/O delay constraints for the target FPGA [9].

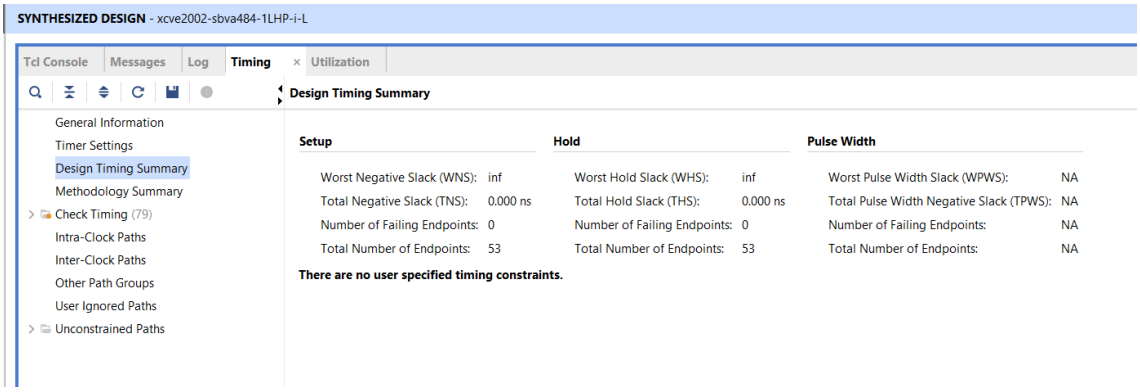


Figure 7.5: Vivado Design Timing Summary — Setup, Hold, and Pulse Width reports.

7.3.4 Device Floorplan

Figure 7.6 shows the implemented design mapped onto the target device floorplan. Given the very low logic utilization reported in Table 11, only a small fraction of the available fabric (concentrated in the lower-left clock region) is actually used by this design.

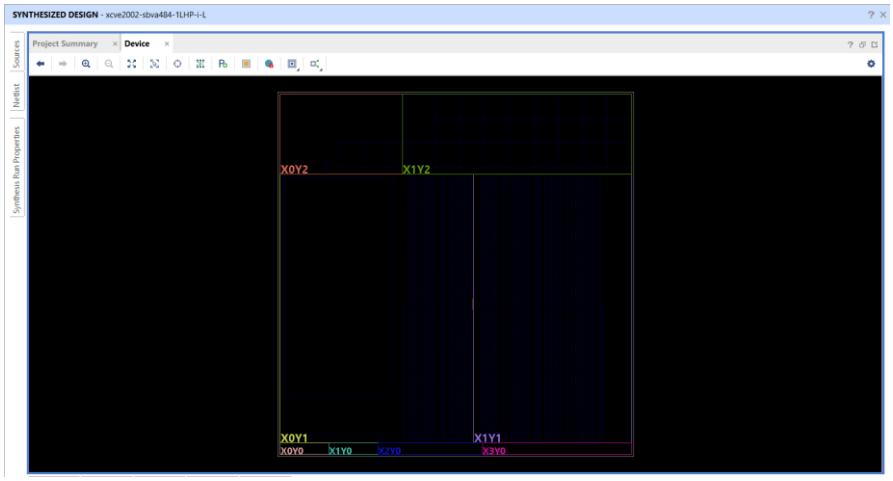


Figure 7.6: Device view showing the physical placement footprint of the synthesized design on the xcve2002-sbva484-1LHP-i-L device.

7.4 Physical Design Implementation using OpenLane 2

7.4.1 Objective and Flow Overview

The FPGA-oriented synthesis of Section 7.3 was run without timing constraints, which is why Vivado reported an "infinite" worst slack. To obtain a real timing and layout result, the same RTL was also taken through a complete ASIC RTL-to-GDSII flow using OpenLane 2 (Classic flow, 78 steps) on the open-source SkyWater SKY130A process with the sky130_fd_sc_hd standard-cell library. The flow performs Yosys/ABC logic synthesis, floorplanning, tap-cell and power-grid generation, I/O pin placement, global and detailed placement, clock-tree synthesis, resizer-based timing and slew repair, global and detailed routing, parasitic extraction, multi-corner static timing analysis (OpenSTA), filler insertion, Magic and KLayout design-rule checks, Netgen layout-versus-schematic comparison, and antenna checks, and finally writes the GDSII layout [10], [11], [12], [13].

7.4.2 Design Preparation

Only the five synthesizable modules (environment_monitor, decision_logic, generic_timer, irrigation_fsm and smart_irrigation_top) were supplied to the flow; the testbench was kept outside the source directory. Before the first run, the RTL was linted with Verilator using the -Wall option, which reported three warnings that were fixed [14]:

- **Missing newline at end of file** – a trailing newline was added to the source files.
- **Unused input inhibit_reason in the FSM** – the port was never read inside the FSM, so it was removed from the FSM and from its instantiation in the top

module. The inhibit_reason output of the top module is still driven directly by the Decision Logic module.

- **Incomplete case statement in the FSM output logic** – an explicit default branch (pump, timer_enable and status all driven to zero) was added to cover the four unused values of the 3-bit state register.

The die size was set manually. The design contains only about ninety logic cells but 54 I/O pins, so an automatically sized die would be too small to place the pins at the minimum spacing. The user-specified configuration of the final run is summarized in Table 12.

Table 12: OpenLane 2 configuration of the final run (parameters not listed were left at their defaults)

Parameter	Value	Purpose
DESIGN_NAME	smart_irrigation_top	Top module
CLOCK_PORT / CLOCK_PERIOD	clk / 5 ns	Clock definition (200 MHz target)
FP_SIZING / DIE_AREA	absolute / 120 × 120 µm	Manual die size (pin-limited design)
SYNTH_STRATEGY	DELAY 0	Delay-oriented ABC mapping script
DESIGN_REPAIR_MAX_SLEW_PCT	40	Extra margin for slew repair
DESIGN_REPAIR_MAX_CAP_PCT	40	Extra margin for capacitance repair
PDK / standard cells	sky130A / sky130_fd_sc_hd	Process and cell library

7.4.3 Timing Constraints

No custom SDC file was provided, so OpenLane applied its generic fallback constraints. For the 5 ns run these are: a 5 ns clock on the clk port, 0.25 ns clock uncertainty, 0.15 ns clock transition, input and output delays of 20% of the clock period (1.0 ns), and a 0.0334 pF load on every output. All timing results in this chapter are therefore relative to this generic constraint set, which treats every input and output as if it were synchronous to the clock, including the purely combinational input-to-output paths of the Decision Logic.

7.4.4 Effect of the Synthesis Strategy

In the first sweep runs, which used the default area-oriented ABC script (AREA 0), all runs that closed timing still reported three max-slew violations at the slowest

corner (ss, 100 °C, 1.60 V). All three belonged to a single net driven by a 5-input AND-OR-INVERT gate (sky130_fd_sc_hd__a2111oi), whose pull-up network consists of four series PMOS transistors and therefore produces a slow rising edge: 0.968 ns against a limit of 0.75 ns. Raising the slew and capacitance repair margins to 40% reduced the number of violations only from four to three, so the cause was the mapped logic structure rather than the repair settings.

Selecting the delay-oriented script (SYNTH_STRATEGY = "DELAY 0") changes the technology-mapping step from ABC's area mapper (amap) to its delay-oriented mapper (map). As shown in Table 13, the wide AOI gates disappeared and were replaced by a larger number of smaller gates with shorter pull-up stacks. All slew violations vanished, at the cost of about 11 additional cells and roughly 5% more layout area.

Table 13: Effect of the ABC synthesis strategy on the synthesized netlist (6 ns runs, identical repair margins)

Metric	**AREA 0 (default)**	**DELAY 0**
a2111oi (5-input AOI) cells	1	0
a211oi cells	1	0
a21oi cells	4	7
o21ai cells	2	4
a311oi cells	0	2
Combinational logic cells (post-layout)	80	90
Synthesized cell area (Yosys)	832 µm²	963 µm²
Post-layout instance area (6 ns run)	1314 µm²	1378 µm²
Max-slew violations (6 ns run)	3	0

The improvement is a result of how the mapper happened to restructure this particular logic, not a guaranteed property of the DELAY strategy, so the slew check should be repeated after any RTL change.

7.4.5 Clock-Period Sweep

The flow was repeated at several clock periods. Because the generic constraints scale the input and output delays with the clock period, the critical paths in these runs are the combinational input-to-output paths of the Decision Logic (for example moisture[3] to inhibit_reason[0]) rather than register-to-register paths; the reported limit is therefore a property of the constraint set as much as of the design.

Table 14: Clock-period sweep (worst slack over all nine PVT corners; DRC, LVS and antenna checks were clean in every run)

Run	Period	Strategy	Setup WS (ns)	Hold WS (ns)	Slew viol.	Area (µm²)	Power (µW)
clk_10	10 ns	AREA 0	+2.79	+0.123	3	1304	193
clk_7	7 ns	AREA 0	+1.78	+0.121	3	1314	276
clk_6	6 ns	AREA 0	+0.79	+0.123	3	1314	322
clk_6_r2	6 ns	DELAY 0	+1.16	+0.119	0	1378	326
clk_5	5 ns	AREA 0	−0.059 (fails)	+0.123	3	1320	387
clk_5_r2 (final)	**5 ns**	**DELAY 0**	**+0.18**	**+0.119**	**0**	**1378**	**391**

Two trends are visible. First, power scales almost linearly with frequency (193 µW at 10 ns, 391 µW at 5 ns), as expected for dynamic power. Second, with the default synthesis strategy the 5 ns run fails setup by 0.059 ns on an input-to-output path, whereas the DELAY 0 netlist closes at 5 ns. An attempt at 4 ns did not close timing (it failed hold on an input-to-register path), so 5 ns is taken as the final operating point of this implementation under the generic constraints.

7.4.6 Final Implementation Results (5 ns, DELAY 0)

Table 15: Summary of the final implementation

Quantity	Result
Die area / core area	14,400 µm² (120 × 120 µm) / 10,630 µm²
Standard-cell area / utilization	1,377.6 µm² / 12.96%
Placed instances (excluding fill)	318 – 152 tap cells, 11 flip-flops, 57 multi-input gates, 17 inverters, 14 buffers, 4 clock buffers, 63 timing-repair buffers
Filler cells	919
Routed wirelength / longest net	2,992 µm / 123.9 µm
Total power	391 µW (internal 275 µW, switching 116 µW, leakage 2.3 nW)
Worst setup slack	+0.181 ns (ss, 100 °C, 1.60 V)
Worst hold slack	+0.119 ns (ff, −40 °C, 1.95 V)
Max slew / cap / fanout violations	0 / 0 / 0
Detailed-routing DRC errors per iteration	81 → 20 → 1 → 0
Final DRC (Magic and KLayout) / LVS / antenna	0 / 0 / 0 – all passed

The layout of the final design is shown in Figures 7.7 and 7.8, and the worst slack in each analysis corner is given in Table 16 [15].

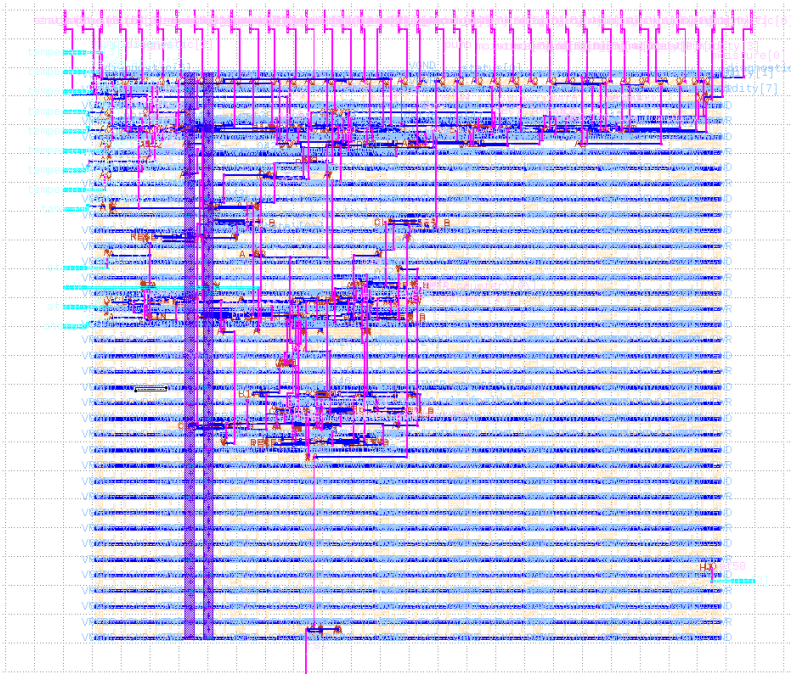


Figure 7.7: Final routed layout of smart_irrigation_top (120 × 120 µm die) viewed in KLayout, showing the local-interconnect, metal-1, via and metal-2 layers. The 54 I/O pins are placed along the die edges.

Table 16: Worst setup and hold slack in each of the nine analysis corners (final run)

Corner	Setup WS (ns)	Hold WS (ns)
max_ff_n40C_1v95	1.890	0.124
max_tt_025C_1v80	1.408	0.344
max_ss_100C_1v60	0.181	0.879
nom_ff_n40C_1v95	1.905	0.122
nom_tt_025C_1v80	1.432	0.341
nom_ss_100C_1v60	0.228	0.872
min_ff_n40C_1v95	1.919	0.119
min_tt_025C_1v80	1.454	0.336
min_ss_100C_1v60	0.265	0.866

As expected, setup timing is limiting at the slow (ss) corners and hold timing at the fast (ff) corners.

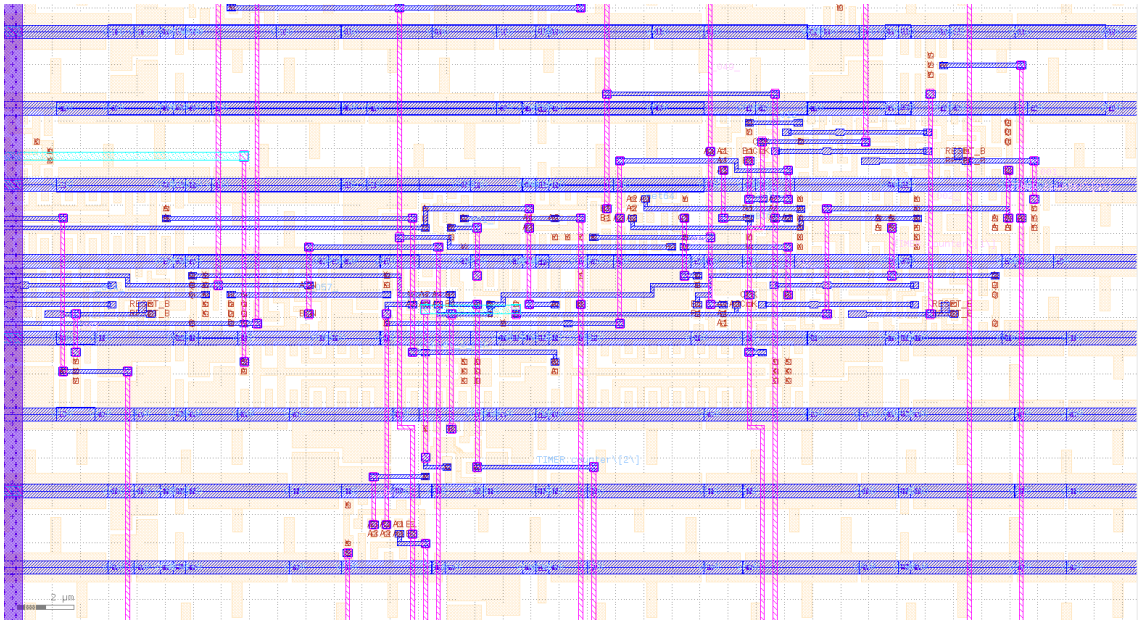


Figure 7.8: Zoomed view of the standard-cell area showing metal-1 power rails, cell-level routing on metal-1 and the vertical metal-2 routes.

7.4.7 Post-Layout (Gate-Level) Verification

To confirm that physical implementation did not change the behaviour of the design, the same self-checking testbench of Section 7.1 was run against the final post-layout netlist (final/nl/smart_irrigation_top.nl.v) together with the SKY130 standard-cell simulation models, using Icarus Verilog in zero-delay functional mode (-DFUNCTIONAL -DUNIT_DELAY=#0) [16].

The first gate-level run passed 7 of 8 checks. The failing check was the first one, immediately after reset (expected state IDLE, observed RESET), while the identical RTL simulation passed. The cause was a race in the testbench rather than a design error: the reset_dut task released rst in the same simulation time step as a clock edge. In the RTL simulation the FSM happened to sample rst = 0 at that edge and moved to IDLE one cycle early, whereas in the netlist the reset signal propagates through the input buffer and inverter in additional delta cycles, so the flip-flops still saw reset asserted at that edge and left RESET one clock later.

The testbench was corrected by releasing reset 1 ns after the clock edge (#1 rst = 0) and delaying the reset-complete messages by 1 ns each, so that the first check now samples at 38 ns instead of 35 ns, safely after the outputs settle. With this change both the RTL simulation and the gate-level simulation of the final netlist report:

```
Tests Passed : 8
Tests Failed : 0
ALL TESTS PASSED
```

```
FSM Coverage
RESET     : 1
IDLE      : 1
WATERING  : 1
FAULT     : 1
```

This simulation is purely functional (zero delay) and covers the eight directed scenarios; it does not include SDF back-annotated timing and is not a formal equivalence check. It does show that, for all scenarios tested, the implemented netlist behaves identically to the RTL.

7.4.8 Observations and Limitations

- **Constant output state[2].** The state register is 3 bits wide but only four states are used, so state[2] is always zero and the synthesis tool ties it to a constant; the timing report lists it as the single unconstrained endpoint.
- **Unused input humidity[0].** The comparison humidity < 70 is equivalent to humidity[7:1] < 35 because 70 is even, so the least-significant bit has no effect on the logic and its pin drives no gate.
- **Thin setup margin.** The 5 ns result has only +0.18 ns of setup slack at the slowest corner and depends on the generic constraint set. A custom SDC with realistic input and output delays would be needed to separate the register-to-register limit from the input-to-output limit.
- **Low utilization.** Logic occupies about 13% of the core because the die size is dictated by the 54 I/O pins rather than by the logic, which is an artefact of implementing the controller as a stand-alone chip. Integrated as a block inside a larger SoC the area would be far smaller.
- **Mapping-dependent slew result.** The removal of the slew violation depends on the structure chosen by the mapper (Section 7.4.4) and should be re-verified after any RTL modification.

7.4.9 Summary

The irrigation controller was successfully taken from RTL to a manufacturing-ready GDSII layout on SKY130 with zero DRC, LVS and antenna violations. It meets setup and hold timing at 5 ns (200 MHz) across all nine PVT corners under the generic constraints, occupies 1,378 μm² of standard-cell area and consumes about 391 μW at 200

MHz. The post-layout netlist reproduces the RTL behaviour on all eight verification scenarios, and the process exposed and corrected a clock-edge race in the original testbench, strengthening the verification environment of Section 7.1.

8. KEY LEARNINGS AND SKILLS ACQUIRED

The internship project provided hands-on experience across the complete path from a sensor-driven embedded prototype to a verified and laid-out digital hardware block. The principal technical and professional competencies developed are summarized below.

8.1 Technical Knowledge

8.1.1 Embedded Systems and IoT Development

- Interfacing analog and digital sensors (capacitive soil-moisture sensor, DHT11, rain sensor and HC-SR04 ultrasonic sensor) with the ESP32, and switching a pump through a relay module.
- Calibrating and filtering sensor readings: ADC averaging, mapping raw values to percentages, and converting echo distance to tank level.
- Developing timer-driven firmware with Wi-Fi supervision and Blynk virtual-pin callbacks for remote monitoring and manual override.
- Configuring cloud dashboards, historical graphs and event notifications on the Blynk platform.

8.1.2 Digital Design and RTL Modelling

- Partitioning a control algorithm into reusable Verilog modules with ANSI-style ports and parameters.
- Designing a Moore FSM, priority-encoded combinational logic and a parameterized counter, and avoiding unintended latches through default assignments.
- Cleaning RTL against Verilator -Wall lint warnings.

8.1.3 Functional Verification

- Writing a self-checking testbench with automatic PASS/FAIL reporting and FSM state-coverage tracking.
- Debugging a clock-edge race that appeared only in gate-level simulation, and understanding how event ordering affects RTL and netlist simulation.

8.1.4 Synthesis and Physical Design

- Running synthesis in Vivado and reading its utilization and timing reports.
- Taking a design from RTL to GDSII with OpenLane 2 on the SKY130 process: configuration, die sizing for a pin-limited design, and interpreting multi-corner static timing analysis.
- Tracing a max-slew violation to a specific mapped gate and removing it by changing the synthesis strategy.
- Checking signoff results (DRC, LVS, antenna) and simulating the post-layout netlist against the original testbench.

8.2 Practical and Analytical Skills

- Comparing design alternatives with measured data (clock-period sweep, area-oriented versus delay-oriented synthesis) instead of assumptions.
- Reading tool logs and reports to trace a result back to its cause.
- Working in a Linux (WSL2) tool flow and keeping runs reproducible through configuration files and run tags.
- Stating the assumptions and limits of each result, such as the generic timing constraints and zero-delay gate-level simulation.

8.3 Professional Competencies

- Technical documentation: organizing the work into a structured report with tables, figures, references and appendices.
- Treating lint warnings, tool warnings and a failing verification run as feedback and resolving each systematically.

9. RESULTS AND DISCUSSION

This chapter consolidates the main results of the two implementation phases and compares the IoT and RTL realizations of the same irrigation control algorithm.

9.1 Summary of Key Results

Table 17 summarizes the key results of the project. The IoT prototype was validated on a live plant pot and water tank, while the RTL design was verified by simulation, synthesized in Vivado, and implemented through an ASIC RTL-to-GDSII flow.

Table 17: Summary of key results of the project

Aspect	Result
IoT prototype	Automatic and manual irrigation on an ESP32 with cloud monitoring (Blynk web and mobile) and tank-empty and low-moisture notifications
RTL functional verification	8 of 8 self-checking tests passed, with 100% FSM state coverage
Vivado synthesis (xcve2002 reference part)	25 LUTs, 12 flip-flops, 53 I/O; run unconstrained, no timing violations reported
OpenLane 2 implementation (SKY130, 5 ns)	Setup slack +0.18 ns, hold slack +0.119 ns across nine PVT corners; zero DRC, LVS and antenna violations
Post-layout area and power	1,378 μm² of standard-cell area; about 391 μW at 200 MHz
Gate-level simulation	8 of 8 tests passed on the final post-layout netlist

9.2 Comparison Between IoT and RTL Implementations

Although both implementations execute logically equivalent irrigation decision-making, they target fundamentally different execution domains, as summarized in Table 18.

Table 18: IoT firmware vs. RTL hardware implementation comparison

Parameter	ESP32 IoT (Phase I)	Verilog RTL (Phase II)
Execution model	Sequential software	Parallel synchronous hardware
Target	Embedded microcontroller	FPGA / ASIC
Programming language	Embedded C++	Verilog HDL
Remote monitoring	Available (Blynk)	Not implemented at RTL level
Cloud support	Native (Blynk)	Would require an external interface (e.g. UART/AXI bridge)
Latency	Software / task-scheduler dependent	Deterministic, hardware clock dependent

Parameter	ESP32 IoT (Phase I)	Verilog RTL (Phase II)
Scalability	Moderate — limited by CPU cycles	High — logic can run fully in parallel
Verification approach	Manual testing with serial monitor / dashboard	Automated, self-checking testbench with coverage tracking
Future integration path	Standalone embedded product	Reusable SoC IP core

Building the software version first provided a fast, low-risk way to validate the irrigation decision thresholds and hardware wiring against a real plant and water tank. Once that logic was confirmed correct in practice, re-expressing it as RTL forced a stricter, more formal specification of the same behaviour — exact priority ordering, exact state encodings, and an automated test suite — which in turn makes the hardware version substantially easier to verify exhaustively than the firmware ever was. This progression from software prototype to verified hardware design is a practical instance of hardware–software co-design, and is the primary motivation for structuring the project in two phases rather than choosing one implementation route from the outset.

10. CONCLUSION AND FUTURE SCOPE

This project has demonstrated the complete development cycle of a smart irrigation controller across two implementation domains. The ESP32-based embedded prototype validated the sensing and control algorithm under real operating conditions — a live plant pot, water tank, and rain sensor — while the RTL implementation proved that the same control behaviour can be realized as synthesizable digital hardware, verified exhaustively through a self-checking testbench, and synthesized with a lightweight resource footprint.

10.1 Major Achievements

- A fully functional ESP32 IoT prototype with live sensor acquisition and relay-driven pump control.
- Both automatic (threshold-based) and manual irrigation modes, safely gated by a tank-empty cut-off in both modes.
- Cloud monitoring and control through the Blynk web dashboard and mobile app, including configured event notifications.
- A modular, four-block Verilog RTL architecture (Environment Monitor, Decision Logic, Generic Timer, Irrigation FSM).
- A self-checking verification environment achieving 8/8 passing test cases and 100% FSM state coverage.
- Successful synthesis in Vivado with a resource footprint of only 25 LUTs and 12 flip-flops (an unconstrained run: no timing violations were reported, but no timing closure is claimed).
- An ASIC RTL-to-GDSII implementation using OpenLane 2 on SKY130 that closes setup and hold timing at 5 ns across nine PVT corners with zero DRC, LVS and antenna violations, verified by gate-level simulation of the final netlist (Section 7.4).

10.2 Future Work

- Adding an AXI-Lite (or similar) register interface so the RTL controller could be integrated as a peripheral in a larger SoC.

- Migrating sensor interfaces to SPI/I²C-based digital sensors for improved accuracy and noise immunity over the current analog/single-wire interfaces.
- Exploring machine-learning-based irrigation prediction using historical Blynk sensor data.
- Solar-powered, fully autonomous field deployment of the IoT prototype.
- Deploying and validating the RTL design on physical FPGA hardware, including a proper timing-constrained (.xdc) implementation run.
- Fabrication hand-off of the OpenLane 2 layout (for example through an open-source multi-project wafer shuttle), preceded by a custom SDC with realistic input/output delays and an SDF-annotated gate-level simulation.

REFERENCES

[1] IEEE Standard for Verilog Hardware Description Language, *IEEE Std 1364-2005*, IEEE, New York, NY, USA, 2006.

[2] Espressif Systems, "ESP32 Series Datasheet," Espressif Systems (Shanghai) Co., Ltd., Shanghai, China. *[Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf*, accessed Oct. 2026.

[3] N. H. E. Weste and D. M. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed. Boston, MA, USA: Pearson, 2010.

[4] Aosong (Guangzhou) Electronics Co., Ltd., "DHT11 Humidity & Temperature Sensor," product datasheet.

[5] Elec Freaks, "Ultrasonic Ranging Module HC-SR04," product datasheet.

[6] Blynk Inc., "Blynk IoT Platform Documentation." *[Online]. Available: https://docs.blynk.io*, accessed Oct. 2026.

[7] S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2003.

[8] AMD, "Vivado Design Suite User Guide: Synthesis (UG901)," AMD Inc., San Jose, CA, USA, 2025.

[9] AMD, "Vivado Design Suite User Guide: Implementation (UG904)," AMD Inc., San Jose, CA, USA, 2025.

[10] "OpenLane 2 Documentation." *[Online]. Available: https://openlane2.readthedocs.io*, accessed Oct. 2026.

[11] SkyWater Technology Foundry, "SkyWater Open Source PDK (SKY130) Documentation." *[Online]. Available: https://skywater-pdk.readthedocs.io*, accessed Oct. 2026.

[12] C. Wolf, "Yosys Open SYnthesis Suite." *[Online]. Available: https://yosyshq.net/yosys/*, accessed Oct. 2026.

[13] T. Ajayi *et al.*, "Toward an open-source digital flow: First learnings from the OpenROAD project," in *Proc. 56th ACM/IEEE Design Automation Conf. (DAC)*, Las Vegas, NV, USA, Jun. 2019.

[14] W. Snyder *et al.*, "Verilator." *[Online]. Available: https://www.veripool.org/verilator/*, accessed Oct. 2026.

[15] M. Köfferlein, "KLayout – layout viewer and editor." *[Online]. Available: https://www.klayout.de*, accessed Oct. 2026.

[16] S. Williams, "Icarus Verilog." *[Online]. Available: https://steveicarus.github.io/iverilog/*, accessed Oct. 2026.

APPENDICES

The complete, unabridged source code for both implementation phases is reproduced in the following appendices for reference. The Wi-Fi credentials and Blynk authentication token in Appendix A have been redacted and replaced with placeholder strings, in line with the security note in Section 5.4.

Appendix A – Complete Arduino Source Code (test.ino)

ESP32 firmware implementing sensor acquisition, automatic/manual irrigation control, and Blynk cloud synchronization, as described in Chapter 5.

```
#define BLYNK_TEMPLATE_ID "TMPL30obLR8fC"
#define BLYNK_TEMPLATE_NAME "Smart Irrigation and Monitoring System"
#define BLYNK_AUTH_TOKEN "YOUR_BLYNK_AUTH_TOKEN"

#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <DHT.h>

char ssid[] = "YOUR_WIFI_SSID";
char pass[] = "YOUR_WIFI_PASSWORD";

#define MOISTURE_PIN 34
#define RAIN_PIN 27
#define RELAY_PIN 26
#define TRIG_PIN 18
#define ECHO_PIN 19

#define DHTPIN 4
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

BlynkTimer timer;

#define TANK_HEIGHT 20
#define MIN_TANK_LEVEL 10
#define DRY_VALUE 1700
#define WET_VALUE 900

#define LOW_THRESHOLD 20
#define HIGH_THRESHOLD 60

bool pumpState = false;
bool manualMode = false;
bool tankAlertSent = false;
bool pumpControl = false;

void pumpON()
{
    digitalWrite(RELAY_PIN, HIGH);   // active LOW relay
    pumpState = true;
}

void pumpOFF()
{
    digitalWrite(RELAY_PIN, LOW);
    pumpState = false;
}

BLYNK_WRITE(V6)
{
    manualMode = param.asInt();
}

BLYNK_WRITE(V5)
{   pumpControl = param.asInt();
    if(manualMode)
    {
        if(param.asInt())
            pumpON();
        else
            pumpOFF();
    }
}

float getDistance()
{
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);

    digitalWrite(TRIG_PIN, HIGH);
```

```
    delayMicroseconds(10);

    digitalWrite(TRIG_PIN, LOW);

    long duration =
        pulseIn(ECHO_PIN, HIGH, 30000);

    float distance =
        duration * 0.034 / 2.0;

    return distance;
}

void sendSensorData()
{
    // Moisture

    long total = 0;

for(int i=0;i<10;i++)
{
    total += analogRead(MOISTURE_PIN);
    delay(5);
}

int rawValue = total / 10;

float distance = getDistance();
int waterLevel =
    ((TANK_HEIGHT - distance) * 100)
    / TANK_HEIGHT;

waterLevel =
    constrain(waterLevel,
              0,
              100);

    int moisturePercent =
        map(rawValue,
            DRY_VALUE,
            WET_VALUE,
            0,
            100);
     moisturePercent =
    constrain(moisturePercent,
              0,
              100);

    // DHT11

    float humidity =
        dht.readHumidity();

    float temperature =
        dht.readTemperature();


    // Rain

    bool rainDetected =
        (digitalRead(RAIN_PIN) == LOW);

    if(!manualMode)
    {
        if(moisturePercent < LOW_THRESHOLD && !rainDetected && waterLevel > MIN_TANK_LEVEL && !pumpState)
        {
            pumpON();
        }

        if(moisturePercent > HIGH_THRESHOLD &&
           pumpState)
        {
            pumpOFF();
        }
    }
    else if (manualMode && !pumpControl) {
      pumpOFF();
      }
    if(waterLevel <= MIN_TANK_LEVEL)
{
    pumpOFF();
```

```
    if(!tankAlertSent)
    {
        Blynk.logEvent(
            "tank_empty",
            "Water tank is empty");

        tankAlertSent = true;
    }
}
else
{
    tankAlertSent = false;
}

    Blynk.virtualWrite(V0, moisturePercent);

    Blynk.virtualWrite(V1, temperature);

    Blynk.virtualWrite(V2, humidity);

    Blynk.virtualWrite(V3,
                       rainDetected ? 1 : 0);

    Blynk.virtualWrite(V4,
                       pumpState ? 1 : 0);

    Blynk.virtualWrite(V7, waterLevel);

    Serial.print("Moisture: ");
    Serial.print(rawValue);
    Serial.print("  ");
    Serial.print(moisturePercent);

    Serial.print("%  Temp: ");
    Serial.print(temperature);

    Serial.print("C  Humidity: ");
    Serial.print(humidity);

    Serial.print("%  Rain: ");
    Serial.print(rainDetected);

    Serial.print("  Tank: ");
    Serial.print(distance);
Serial.print("  ");
Serial.print(waterLevel);
Serial.print("%");

    Serial.print("  Pump: ");
    Serial.println(pumpState);
}

void checkWiFi()
{
    if(WiFi.status() != WL_CONNECTED)
    {
        Serial.println("Reconnecting WiFi...");

        WiFi.disconnect();
        WiFi.reconnect();
    }
}

void setup()
{
    Serial.begin(115200);

    pinMode(RAIN_PIN, INPUT);

    pinMode(RELAY_PIN, OUTPUT);

    pinMode(TRIG_PIN, OUTPUT);
pinMode(ECHO_PIN, INPUT);

    pumpOFF();

    dht.begin();

    Blynk.begin(
        BLYNK_AUTH_TOKEN,
```

```
        ssid,
        pass);

    timer.setInterval(
        2000L,
        sendSensorData);

    timer.setInterval(
    30000L,
    checkWiFi);
}

void loop()
{
    Blynk.run();
    timer.run();
}
```

Appendix B – environment_monitor.v

Combinational module that converts raw moisture/temperature/humidity readings into the dry, wet, hot, and low_humidity flags described in Section 6.2.

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 15.07.2026 17:21:06
// Design Name:
// Module Name: environment_monitor
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


module environment_monitor #(
    parameter LOW_MOISTURE  = 20,
    parameter HIGH_MOISTURE = 70,
    parameter HIGH_TEMP     = 30,
    parameter LOW_HUMIDITY  = 70
)(
    input [7:0] moisture,
    input [7:0] temperature,
    input [7:0] humidity,
    output dry,
    output wet,
    output hot,
    output low_humidity);

assign dry          = (moisture < LOW_MOISTURE);

assign wet          = (moisture > HIGH_MOISTURE);

assign hot          = (temperature > HIGH_TEMP);

assign low_humidity = (humidity < LOW_HUMIDITY);


endmodule
```

Appendix C – decision_logic.v

Priority-encoded combinational module that produces watering_request, inhibit_reason, and env_status, as described in Section 6.3.

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 15.07.2026 17:50:18
// Design Name:
// Module Name: decision_logic
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


module decision_logic(
    input dry,
    input hot,
    input low_humidity,
    input rain,
    input tank_empty,
    output reg watering_request,
    output reg [2:0] inhibit_reason,
    output wire [3:0] env_status
    );

assign env_status = {
    rain,
    low_humidity,
    hot,
    dry
};

localparam NO_INHIBIT      = 3'b000;
localparam RAIN_DETECTED   = 3'b001;
localparam TANK_EMPTY      = 3'b010;
localparam TEMP_TOO_LOW    = 3'b011;
localparam HUMIDITY_HIGH   = 3'b100;
localparam SOIL_NOT_DRY    = 3'b101;


//------------------------------------------------------------
// Decision Logic
//------------------------------------------------------------

always @(*)
begin

    // Default outputs

    watering_request = 1'b0;
    inhibit_reason   = NO_INHIBIT;

    //------------------------------------------------------
    // Highest Priority : Tank Empty
    //------------------------------------------------------

    if(tank_empty)
    begin
        watering_request = 1'b0;
        inhibit_reason   = TANK_EMPTY;
    end
```

```
    //---------------------------------------------------------
    // Rain Detected
    //---------------------------------------------------------

    else if(rain)
    begin
        watering_request = 1'b0;
        inhibit_reason   = RAIN_DETECTED;
    end

    //---------------------------------------------------------
    // Temperature Too Low
    //---------------------------------------------------------

    else if(!hot)
    begin
        watering_request = 1'b0;
        inhibit_reason   = TEMP_TOO_LOW;
    end

    //---------------------------------------------------------
    // Humidity Already High
    //---------------------------------------------------------

    else if(!low_humidity)
    begin
        watering_request = 1'b0;
        inhibit_reason   = HUMIDITY_HIGH;
    end

    //---------------------------------------------------------
    // Soil Not Dry
    //---------------------------------------------------------

    else if(!dry)
    begin
        watering_request = 1'b0;
        inhibit_reason   = SOIL_NOT_DRY;
    end

    //---------------------------------------------------------
    // Watering Allowed
    //---------------------------------------------------------

    else
    begin
        watering_request = 1'b1;
        inhibit_reason   = NO_INHIBIT;
    end

end

endmodule
```

Appendix D – generic_timer.v

Parameterized synchronous counter used to enforce a fixed watering duration, as described in Section 6.4.

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 17.07.2026 19:24:46
// Design Name:
// Module Name: generic_timer
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


module generic_timer #(
    parameter COUNT_WIDTH = 32,
    parameter TIMER_COUNT = 100
)
(
    input wire clk,
    input wire rst,

    input wire enable,

    output reg timer_done
);

reg [COUNT_WIDTH-1:0] counter;

always @(posedge clk or posedge rst)
begin

    if(rst)
    begin

        counter <= 0;
        timer_done <= 0;

    end

    else if(enable)
    begin

        if(counter < TIMER_COUNT)
        begin

            counter <= counter + 1;
            timer_done <= 0;

        end

        else
        begin

            timer_done <= 1;

        end

    end

    else
    begin
```

```
        counter <= 0;
        timer_done <= 0;

    end

end
endmodule
```

Appendix E – irrigation_fsm.v

Four-state Moore FSM (RESET / IDLE / WATER / FAULT) that generates the pump and diagnostic outputs, as described in Section 6.5.

```
//------------------------------------------------------------
// Module : irrigation_fsm
// Description : Moore FSM for Smart Irrigation Controller
//------------------------------------------------------------

module irrigation_fsm(
    input wire timer_done,
    input wire clk,
    input wire rst,

    input dry,
    input hot,
    input low_humidity,
    input rain,

    input wire watering_request,
    input wire [2:0] inhibit_reason,
    input wire wet,

    input wire tank_empty,

    input wire manual_mode,
    input wire manual_cmd,

    output reg pump,
    output reg timer_enable,
    output reg [4:0] status,
    output wire [7:0] diagnostic,
    output wire [2:0] state

);

//------------------------------------------------------------
// State Encoding
//------------------------------------------------------------

localparam RESET_ST = 3'd0;
localparam IDLE_ST  = 3'd1;
localparam WATER_ST = 3'd2;
localparam FAULT_ST = 3'd3;

reg [2:0] current_state;
reg [2:0] next_state;

assign state = current_state;

////////////////////////////////////////////////////////////////
// State Register
////////////////////////////////////////////////////////////////

always @(posedge clk or posedge rst)
begin

    if(rst)
        current_state <= RESET_ST;
    else
        current_state <= next_state;

end

////////////////////////////////////////////////////////////////
// Next State Logic
////////////////////////////////////////////////////////////////

always @(*)
begin

    next_state = current_state;

    case(current_state)
```

```
//-------------------------------------------------------
RESET_ST:
//-------------------------------------------------------

    next_state = IDLE_ST;

//-------------------------------------------------------
//IDLE
//-------------------------------------------------------

IDLE_ST:
begin

    if(tank_empty)

        next_state = FAULT_ST;

    else if(manual_mode)

    begin

        if(manual_cmd)

            next_state = WATER_ST;

        else

            next_state = IDLE_ST;

    end

    else

    begin

        if(watering_request)

            next_state = WATER_ST;

        else

            next_state = IDLE_ST;

    end

end

//-------------------------------------------------------
//WATERING
//-------------------------------------------------------

WATER_ST:
begin

    if(tank_empty)

        next_state = FAULT_ST;

    else if(manual_mode)

    begin

        if(manual_cmd)

            next_state = WATER_ST;

        else

            next_state = IDLE_ST;

    end

    else if(timer_done && wet)

        next_state = IDLE_ST;

    else

        next_state = WATER_ST;
```

```
    end

    //-------------------------------------------------------
    //FAULT
    //-------------------------------------------------------

    FAULT_ST:
    begin

        if(tank_empty)

            next_state = FAULT_ST;

        else

            next_state = IDLE_ST;

    end

    default:

        next_state = RESET_ST;

    endcase

end

////////////////////////////////////////////////////////////
// Output Logic (Moore)
////////////////////////////////////////////////////////////

always @(*)
begin

    //-------------------------------------------------------
    //Default Outputs
    //-------------------------------------------------------

    pump   = 1'b0;
    timer_enable = 1'b0;
    status = 5'b00000;

    case(current_state)

    //-------------------------------------------------------
    //RESET
    //-------------------------------------------------------

    RESET_ST:

    begin

        pump   = 1'b0;
        status = 5'b00000;

    end

    //-------------------------------------------------------
    //IDLE
    //-------------------------------------------------------

    IDLE_ST:

    begin

        pump = 1'b0;

        status[1] = 1'b1;

        if(manual_mode)
            status[4] = 1'b1;

    end

    //-------------------------------------------------------
    //WATERING
    //-------------------------------------------------------

    WATER_ST:

    begin
```

```
        pump = 1'b1;
        timer_enable = 1'b1;

        status[0] = 1'b1;
        status[2] = 1'b1;

        if(manual_mode)
            status[4] = 1'b1;

    end

    //-------------------------------------------------------
    //FAULT
    //-------------------------------------------------------

    FAULT_ST:

    begin

        pump = 1'b0;

        status[3] = 1'b1;

    end

    endcase

end

assign diagnostic = {watering_request, timer_done, tank_empty, rain, low_humidity, hot, wet, dry};

endmodule
```

Appendix F – smart_irrigation_top.v

Top-level module instantiating and interconnecting the Environment Monitor, Decision Logic, Generic Timer, and Irrigation FSM, as described in Section 6.6.

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 17.07.2026 21:39:40
// Design Name:
// Module Name: smart_irrigation_top
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


module smart_irrigation_top(
    input wire clk,
    input wire rst,

    input wire [7:0] moisture,
    input wire [7:0] temperature,
    input wire [7:0] humidity,

    input wire rain,
    input wire tank_empty,

    input wire manual_mode,
    input wire manual_cmd,

    output wire pump,
    output wire [4:0] status,
    output wire [2:0] state,
    output wire [2:0] inhibit_reason,
    output wire [3:0] env_status,
    output wire [7:0] diagnostic
    );
wire dry;
wire wet;
wire hot;
wire low_humidity;

wire watering_request;

wire timer_enable;
wire timer_done;

environment_monitor ENV_MON
(
    .moisture(moisture),
    .temperature(temperature),
    .humidity(humidity),

    .dry(dry),
    .wet(wet),
    .hot(hot),
    .low_humidity(low_humidity)
);

decision_logic DECISION
(
    .dry(dry),
    .hot(hot),
    .low_humidity(low_humidity),
```

```
    .rain(rain),
    .tank_empty(tank_empty),

    .watering_request(watering_request),
    .inhibit_reason(inhibit_reason),
    .env_status(env_status)
);

generic_timer
#(
    .COUNT_WIDTH(8),
    .TIMER_COUNT(100)
)
TIMER
(
    .clk(clk),
    .rst(rst),

    .enable(timer_enable),

    .timer_done(timer_done)
);

irrigation_fsm FSM
(
    .clk(clk),
    .rst(rst),

    .watering_request(watering_request),
    .wet(wet),

    .tank_empty(tank_empty),

    .manual_mode(manual_mode),
    .manual_cmd(manual_cmd),

    .timer_done(timer_done),
    .timer_enable(timer_enable),

    .pump(pump),

    .status(status),
    .state(state),
    .dry(dry),
    .hot(hot),
    .low_humidity(low_humidity),
    .rain(rain),
    .inhibit_reason(inhibit_reason),
    .diagnostic(diagnostic)
);

endmodule
```

Appendix G – tb_smart_irrigation.v

Self-checking testbench implementing the eight verification scenarios and FSM coverage tracking described in Section 7.1. This is the updated version (1 ns reset-release offset, Section 7.4.7) used for both the RTL and the gate-level simulations.

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 20.07.2026 16:47:12
// Design Name:
// Module Name: tb_smart_irrigation
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


module tb_smart_irrigation(

    );
reg clk;
reg rst;

reg [7:0] moisture;
reg [7:0] temperature;
reg [7:0] humidity;

reg rain;
reg tank_empty;
reg manual_mode;
reg manual_cmd;
wire pump;

wire [4:0] status;
wire [2:0] state;
wire [2:0] inhibit_reason;
wire [3:0] env_status;

smart_irrigation_top DUT
(

    .clk(clk),
    .rst(rst),

    .moisture(moisture),
    .temperature(temperature),
    .humidity(humidity),

    .rain(rain),
    .tank_empty(tank_empty),

    .manual_mode(manual_mode),
    .manual_cmd(manual_cmd),

    .pump(pump),

    .status(status),

    .state(state),

    .inhibit_reason(inhibit_reason),

    .env_status(env_status)
```

```
);
integer pass_count = 0;
integer fail_count = 0;

reg visited_reset = 0;
reg visited_idle = 0;
reg visited_water =0;
reg visited_fault =0;

always #5 clk = ~clk;

initial
begin
    clk = 0;
    rst = 0;
    moisture = 0;
    temperature = 0;
    humidity = 0;
    rain = 0;
    tank_empty = 0;
    manual_mode = 0;
    manual_cmd = 0;
end

//--------------------------------------------------
// Reset DUT
//--------------------------------------------------

task reset_dut;

begin

    rst = 1;

    repeat(3) @(posedge clk);

    #1 rst = 0;

    @(posedge clk);

    #1 $display("---------------------------------------");
    #1 $display("RESET COMPLETE");
    #1 $display("---------------------------------------");
end
endtask

//--------------------------------------------------
// Set Environment
//--------------------------------------------------

task set_environment;

input [7:0] moist;
input [7:0] temp;
input [7:0] hum;

input rain_in;
input tank_in;

begin

    moisture   = moist;
    temperature = temp;
    humidity    = hum;

    rain       = rain_in;
    tank_empty = tank_in;

    @(posedge clk);

end
endtask

//--------------------------------------------------
// Manual Mode
//--------------------------------------------------

task manual_control;

input mode;
```

```
input cmd;

begin

    manual_mode = mode;
    manual_cmd  = cmd;

    @(posedge clk);

end
endtask

//------------------------------------------------
// Output Checker
//------------------------------------------------

task check_output;

input expectedPump;

input [2:0] expectedState;

begin

    if(pump == expectedPump &&
       state == expectedState)

    begin

        pass_count = pass_count + 1;

        $display("[PASS] Time = %0t", $time);

    end

    else

    begin

        fail_count = fail_count + 1;

        $display("[FAIL] Time = %0t", $time);

        $display("Expected Pump : %b", expectedPump);
        $display("Actual Pump   : %b", pump);

        $display("Expected State: %d", expectedState);
        $display("Actual State  : %d", state);

    end

end
endtask

always @(posedge clk)
begin

    case(state)

    3'd0:
        visited_reset = 1;

    3'd1:
        visited_idle = 1;

    3'd2:
        visited_water = 1;

    3'd3:
        visited_fault = 1;

    endcase
end

initial
begin
//------------------------------------------------
// TEST 1 : RESET
//------------------------------------------------

$display("\n========== TEST 1 : RESET ==========");
```

```
reset_dut();

check_output(
    0,
    3'd1      // IDLE State
);

//------------------------------------------------
// TEST 2 : AUTOMATIC WATERING
//------------------------------------------------

$display("\n========== TEST 2 : DRY SOIL ==========");

set_environment(

    10,     // Moisture

    35,     // Temperature

    40,     // Humidity

    0,      // Rain

    0       // Tank Empty

);

repeat(3) @(posedge clk);

check_output(

    1,

    3'd2     // WATERING

);

//------------------------------------------------
// TEST 3 : TIMER
//------------------------------------------------

$display("\n========== TEST 3 : TIMER ==========");

repeat(110) @(posedge clk);

set_environment(

    85,

    35,

    40,

    0,

    0

);

repeat(2) @(posedge clk);

check_output(

    0,

    3'd1

);

//------------------------------------------------
// TEST 4 : RAIN
//------------------------------------------------

$display("\n========== TEST 4 : RAIN ==========");

set_environment(

    10,

    35,
```

```
    40,

    1,

    0

);

repeat(2) @(posedge clk);

check_output(

    0,

    3'd1

);

//---------------------------------------------------
// TEST 5 : TANK EMPTY
//---------------------------------------------------

$display("\n========== TEST 5 : TANK EMPTY ==========");

set_environment(

    10,

    35,

    40,

    0,

    1

);

repeat(2) @(posedge clk);

check_output(

    0,

    3'd3      // FAULT

);

//---------------------------------------------------
// TEST 6 : RECOVERY
//---------------------------------------------------

$display("\n========== TEST 6 : RECOVERY ==========");

set_environment(

    85,
    35,
    40,
    0,
    0

);

repeat(2) @(posedge clk);

check_output(

    0,
    3'd1

);

//---------------------------------------------------
// TEST 7 : MANUAL MODE
//---------------------------------------------------

$display("\n========== TEST 7 : MANUAL ==========");

manual_control(
```

```
    1,
    1
);
repeat(2) @(posedge clk);
check_output(
    1,
    3'd2
);
manual_control(
    1,
    0
);
repeat(2) @(posedge clk);
check_output(
    0,
    3'd1
);
manual_control(
    0,
    0
);
repeat(2) @(posedge clk);

//------------------------------------------------
// SUMMARY
//------------------------------------------------

$display("\n===================================");
$display("Simulation Summary");
$display("===================================");
$display("Tests Passed : %0d",pass_count);
$display("Tests Failed : %0d",fail_count);
if(fail_count==0)
    $display("ALL TESTS PASSED");
else
    $display("SOME TESTS FAILED");
$display("");
$display("FSM Coverage");
$display("RESET     : %b",visited_reset);
$display("IDLE      : %b",visited_idle);
$display("WATERING  : %b",visited_water);
$display("FAULT     : %b",visited_fault);
$finish;
end
```

```
endmodule
```